AISE4010

Lecture 5

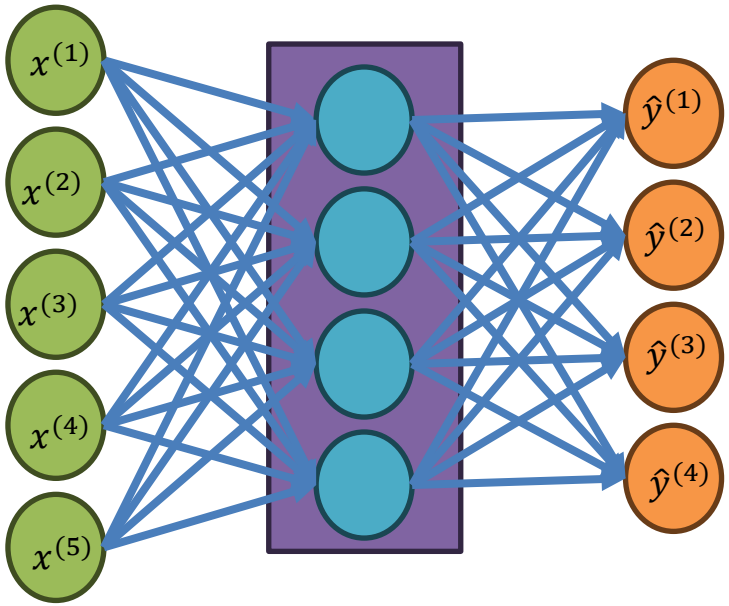
Recurrent Neural Networks

(RNN, LSTM, GRU)

Oct 21, 2026

Recap: Feed Forward Neural Network

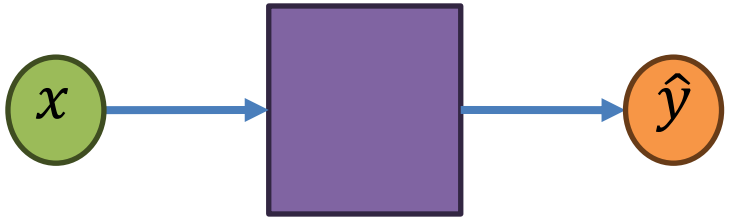
- Memoryless: each input shown to them is processed independently, with no state kept between inputs.



Memoryless models

Example - Image classification:

- Input: one image.
- Output: the probability distribution of classes.
- Need to provide **one** guess (output), and to do that, only need to look at **one** image (input).



CNN Limitations

- Fixed input length: require fixed-length input sequences. If your time series data has varying lengths, you'll need to preprocess (using padding, truncation, or resampling), potentially leading to information loss.

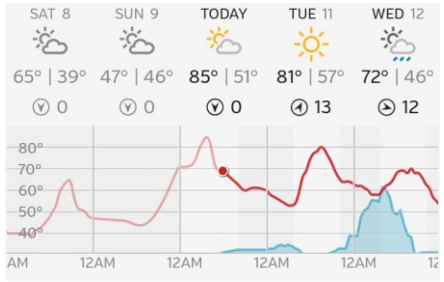
- Limited long-term memory: CNNs are designed for capturing local patterns and struggle to capture long-term dependencies present in some time series.

5.1. Analyze Sequential Data

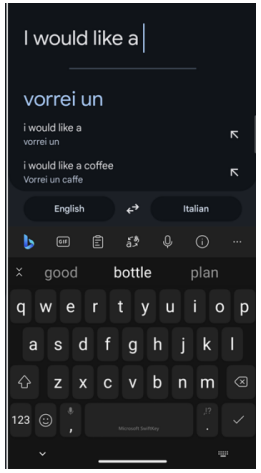
Sequential Models for Sequence Modelling

- How to predict sequential data using a NN with memory and flexible input/output length?
  - CO2 concentration in the process of gas furnace is not only dependent on the current airflow rate but also affected by the process conditions in the preceding steps.
  - In EEG signal the temporal property of the data plays a key role in the pattern recognition of the signal.

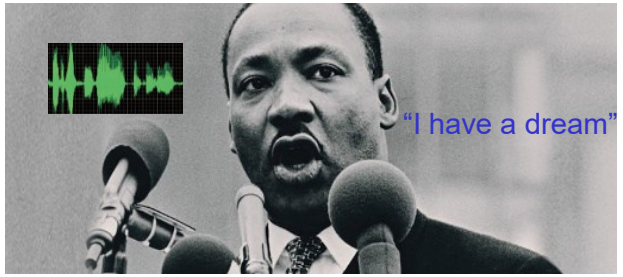




Time Series Forecasting



Machine Translation



Speech to Text

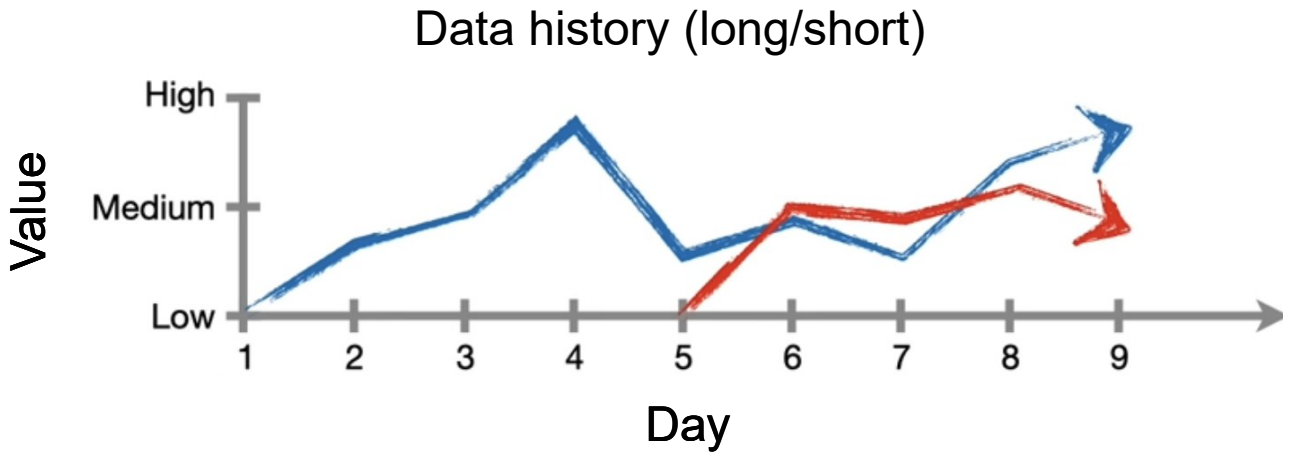


ECG to Diagnostic Label (Normal/Abnormal)

Sequence Modeling - Design Criteria

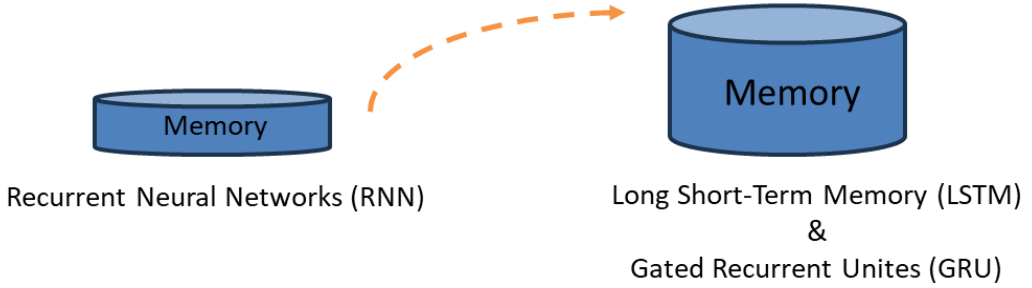
- Needed criteria:
  - Handle variable-length inputs or outputs sequences (a flexible NN in terms of the length of sequence).
  - Track long-term dependencies and maintain information about order
  - Share parameters across sequence

RNNs recognize the data's sequential characteristics and use patterns to predict the next likely scenario.





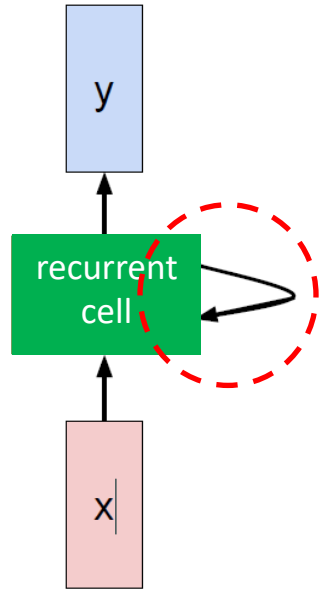
stock market trend



5.2. Recurrent Neural Networks (RNN)

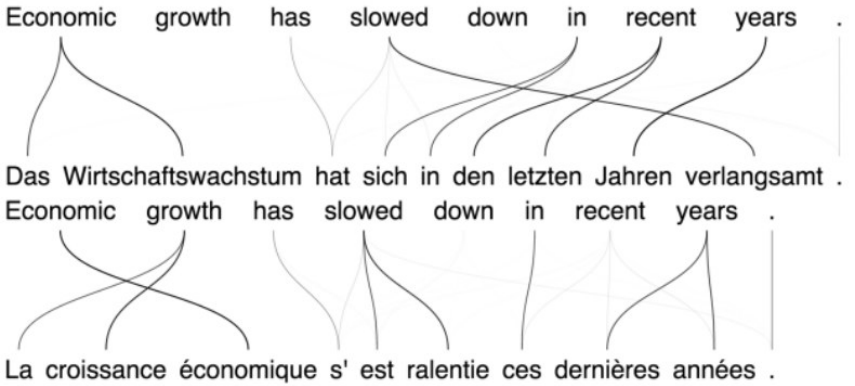
Recurrent Neural Networks (RNN)

- Conventional neural networks have a fixed number of (possibly independent) input dimensions and outputs.
- Sequences often have variable length in which the different items (dimensions) of the sequence are related to one another:
  - Words in a sentence.
  - Values in a time-series
  - Biological sequences

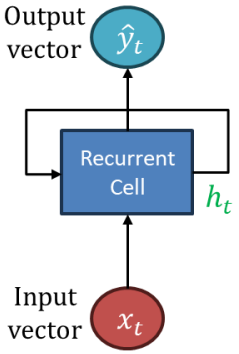


- Recurrent neural networks address such domains.
- Introduce recurrent connection (feedback loop) which allows sequential analysis by learning from data values over time

Recurrent Neural Networks (RNN)



- RNN models sequential data which involves variable length inputs or outputs
- Training data (a sequence) is presented to RNN one at a time



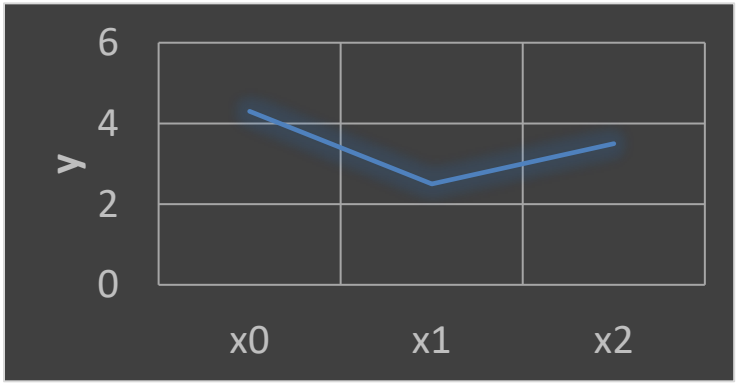
- Takes the input entry and the previous hidden state to compute the output entry

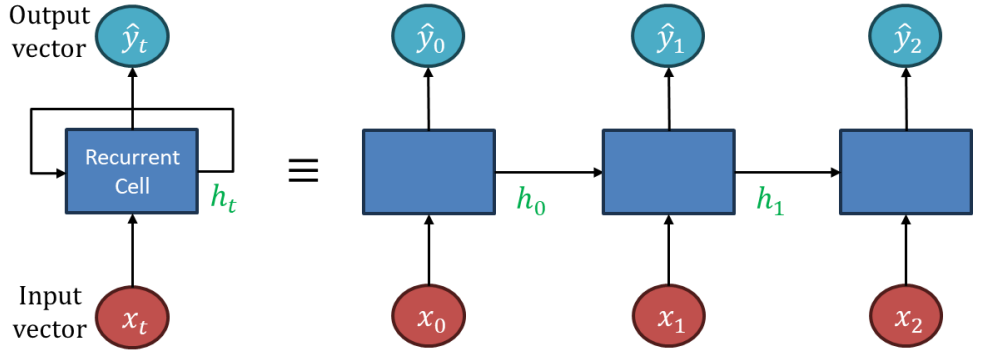
Examples:

- NLP: Natural language processing (sentiment analysis, machine translation, …)
- Image captioning
- Speech recognition
- Protein Structure Prediction
- Weather forecast

RNN Computational Graph

- Instead of imagining that hidden state is being recurrently fed back into the network, it's easier to visualize the process if we unroll the operation into a computational graph that is composed to many time steps.





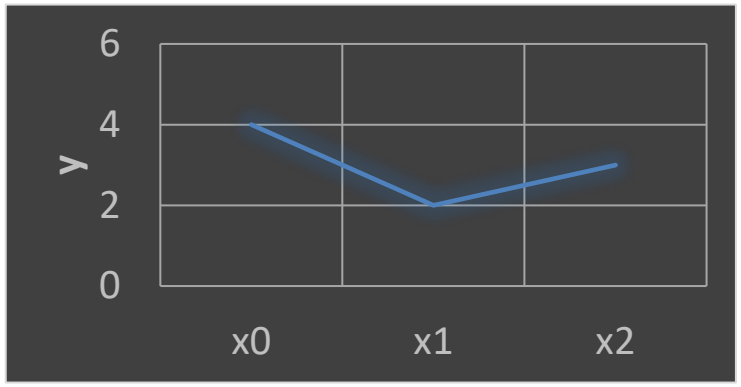
$$\underbrace{\hat{y}_t}_{\text{Output}} = f(\underbrace{x_t}_{\text{Input}}, \underbrace{h_{t-1}}_{\text{Past memory}}) = \boldsymbol{W}_{hy}^T h_t$$

$$h_t = tanh(\boldsymbol{W}_{hh}^T h_{t-1} + \boldsymbol{W}_{xh}^T x_t)$$

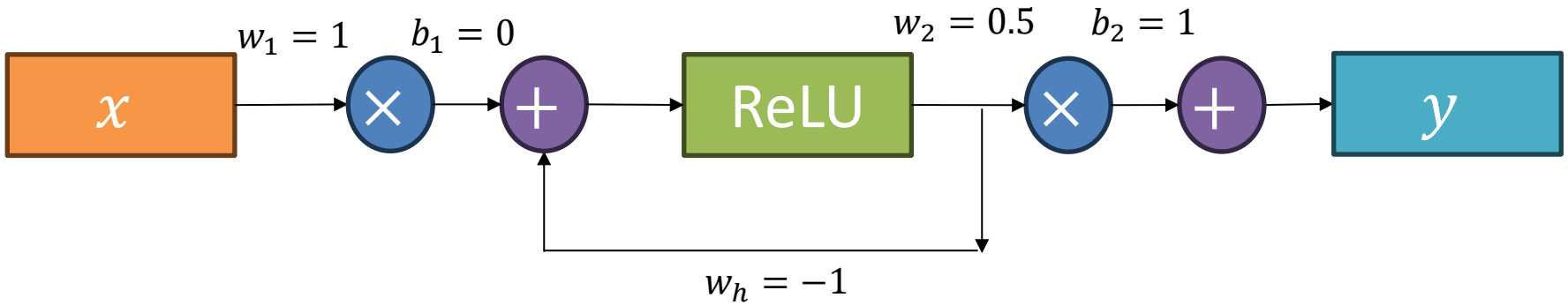
- We use the same set of weight for every time step of the computation.

RNN

- Example

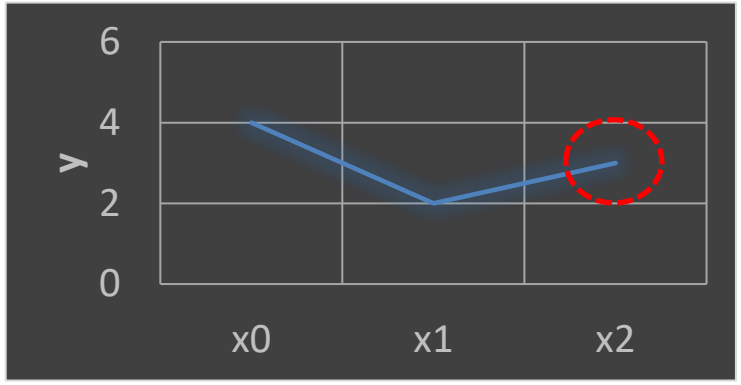


Input	Output
0.5	4
1.5	2
2.5	3

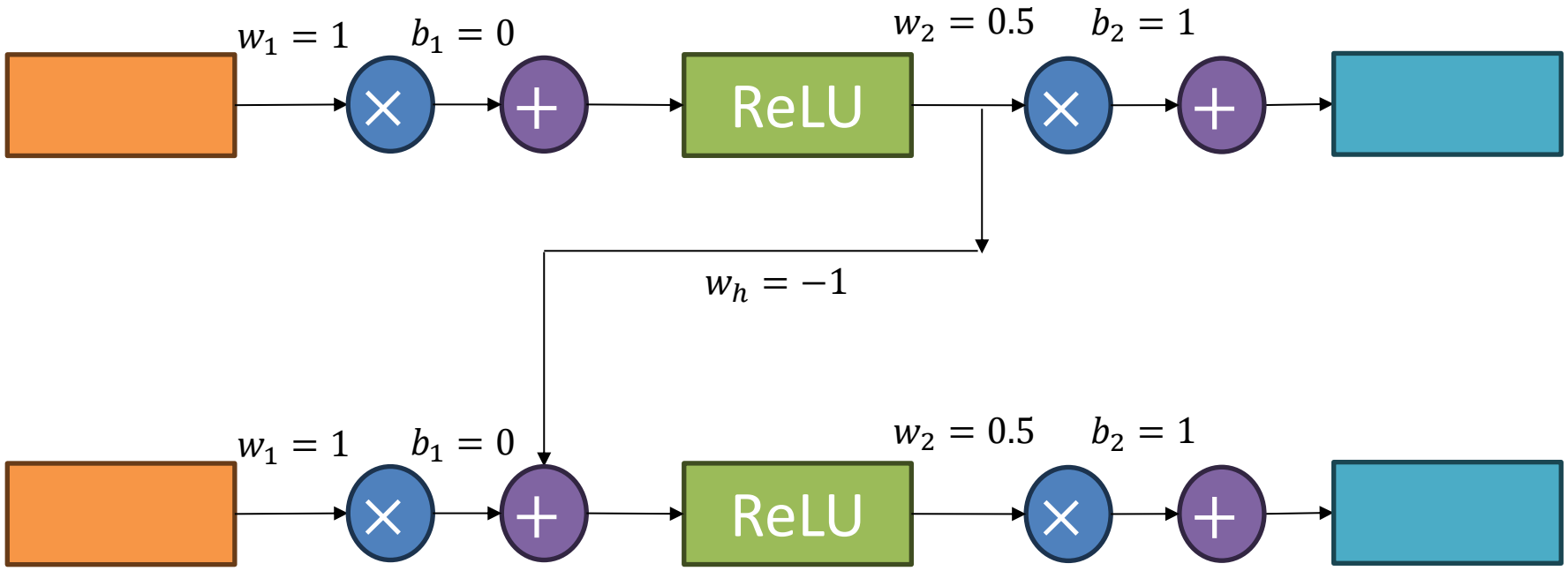


RNN

- Example

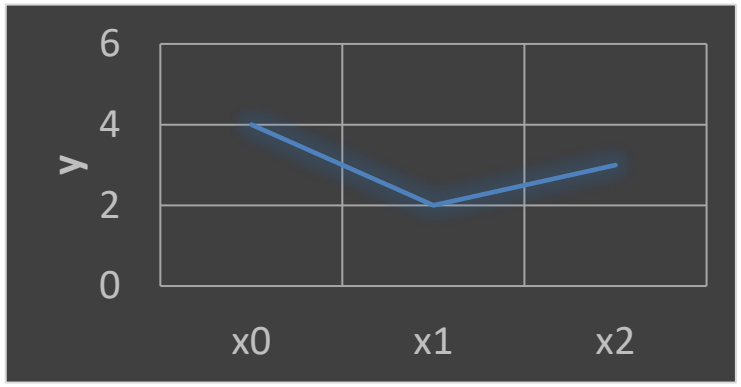


Input	Output
0.5	4
1.5	2
2.5	3



RNN

- Example



Input	Output
0.5	4
1.5	2
2.5	3

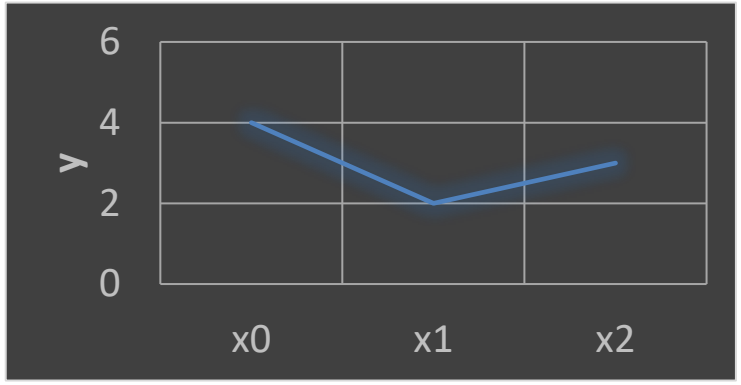


x_0

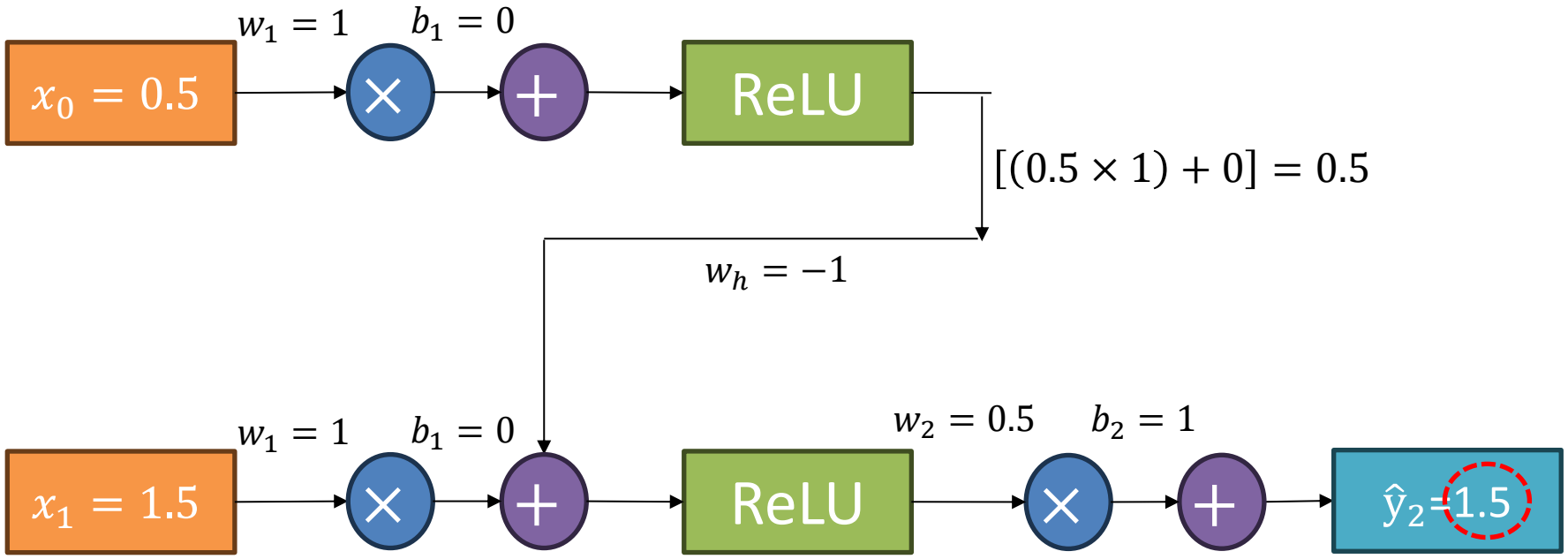
$[(0.5 \times 1) + 0] = 0.5$ → ReLU → $[(0.5 \times 0.5) + 1] = 1.25 = \hat{y}_1$

RNN

- Example



Input	Output
0.5	4
1.5	2
2.5	3



$[(0.5 \times -1) + (1.5 \times 1)] + 0 = 1$ → ReLU → $[(1 \times 0.5) + 1] = 1.5 = \hat{y}_2$

Types of RNN

- Many to One
- One to Many
- Many to Many

Types of RNN

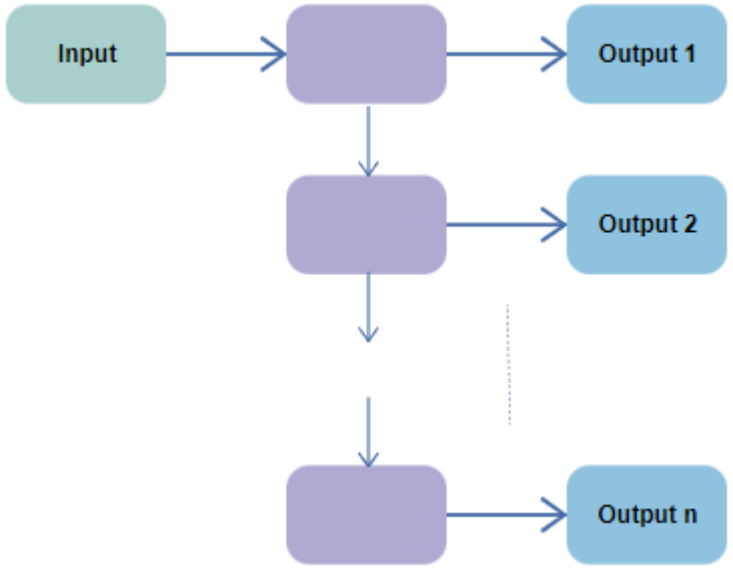
- One to Many

Gives multiple outputs when given a single input.

It takes a fixed input size and gives a sequence of data outputs.

(*e.g. image captioning, music generation*)





Types of RNN

- Many to One

It is used when a single output is required from multiple input units or a sequence of them.

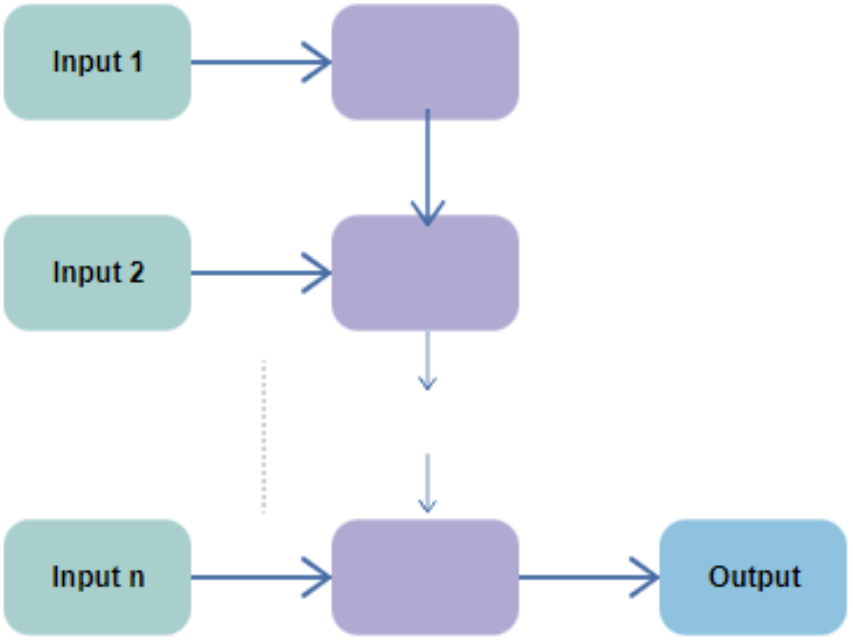
It takes a sequence of inputs to display a fixed output.

(*e.g.* sentiment classification, action prediction)

The Beef Shawarma was **Delicious**! **Best** Shawarma restaurant in Canada



98% positive



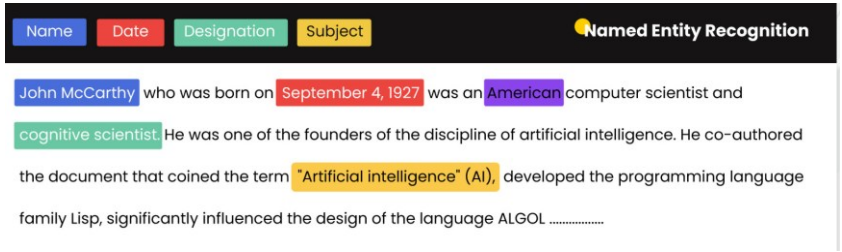
Types of RNN

- Many to Many

It is used to generate a sequence of output data from a sequence of input units:

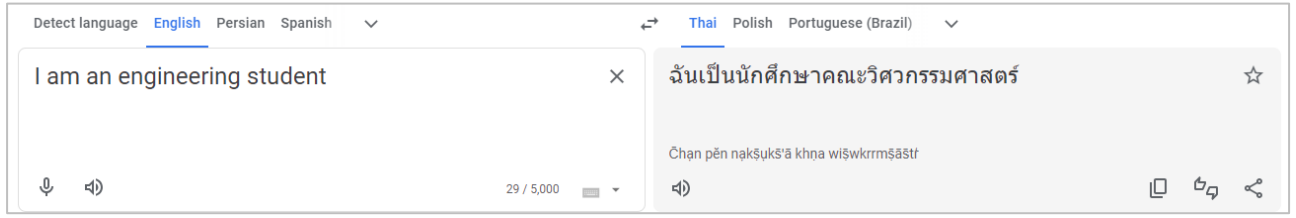
- Equal Unit Size

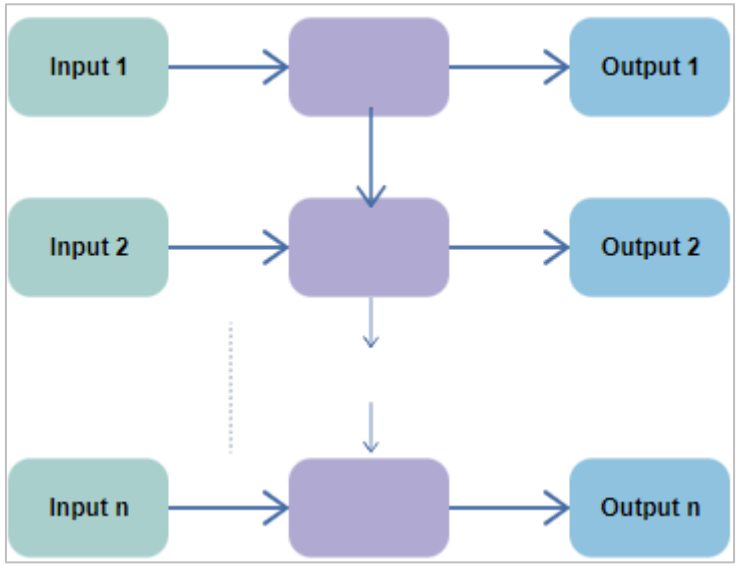
  The number of both the input and output units is the same (e.g., name-entity recognition)

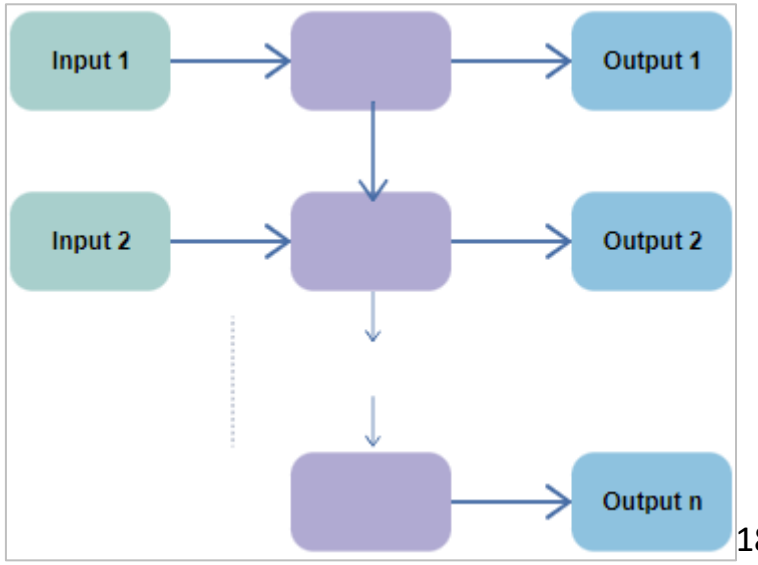


- Unequal Unit Size

  Inputs and outputs have different numbers of units (*e.g., machine translation*)

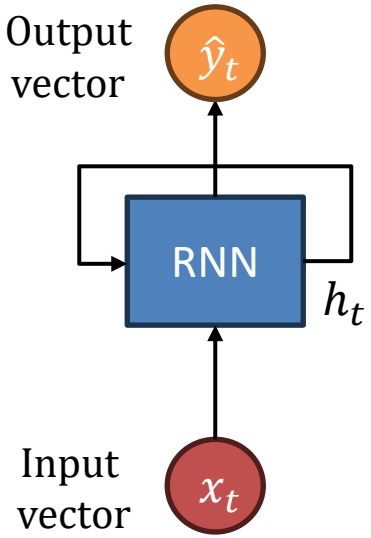






Standard RNN / Vanilla RNN Architecture

- Hidden states serve as the memory for RNN
- The forward pass of a vanilla RNN:
  - The same as that of an MLP with a single hidden layer
  - Except that activations arrive at the hidden layer from both the current external input and the hidden layer activations one step back in time.



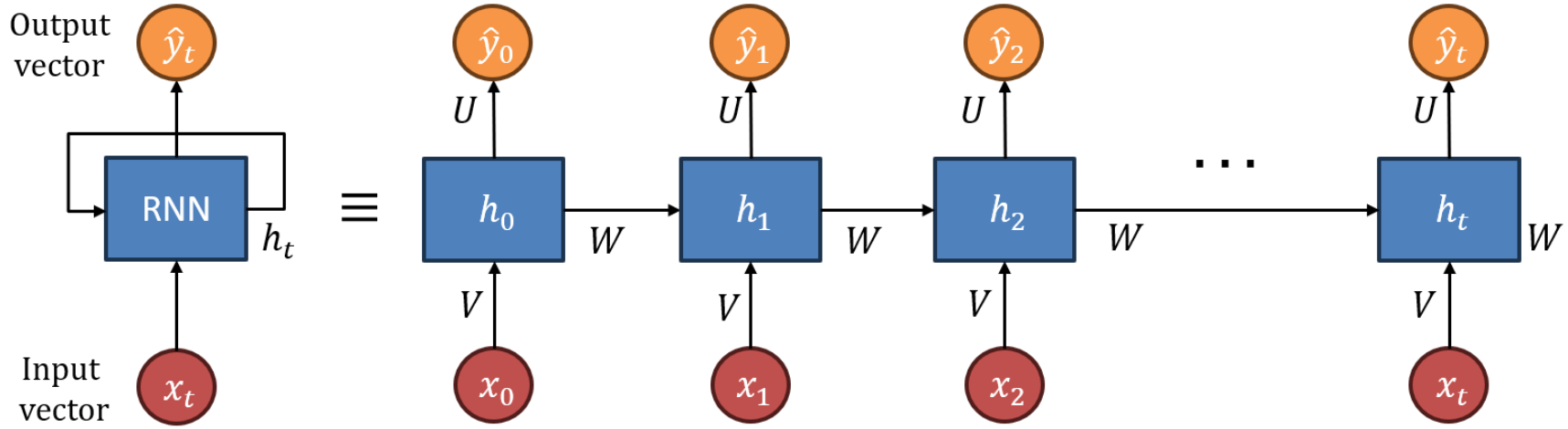
$$\hat{y}_t = \boldsymbol{W}_{hy}^T h_t$$

Updating hidden state $$h_t = tanh(\boldsymbol{W}_{hh}^T h_{t-1} + \boldsymbol{W}_{xh}^T x_t)$$

$$x_t$$

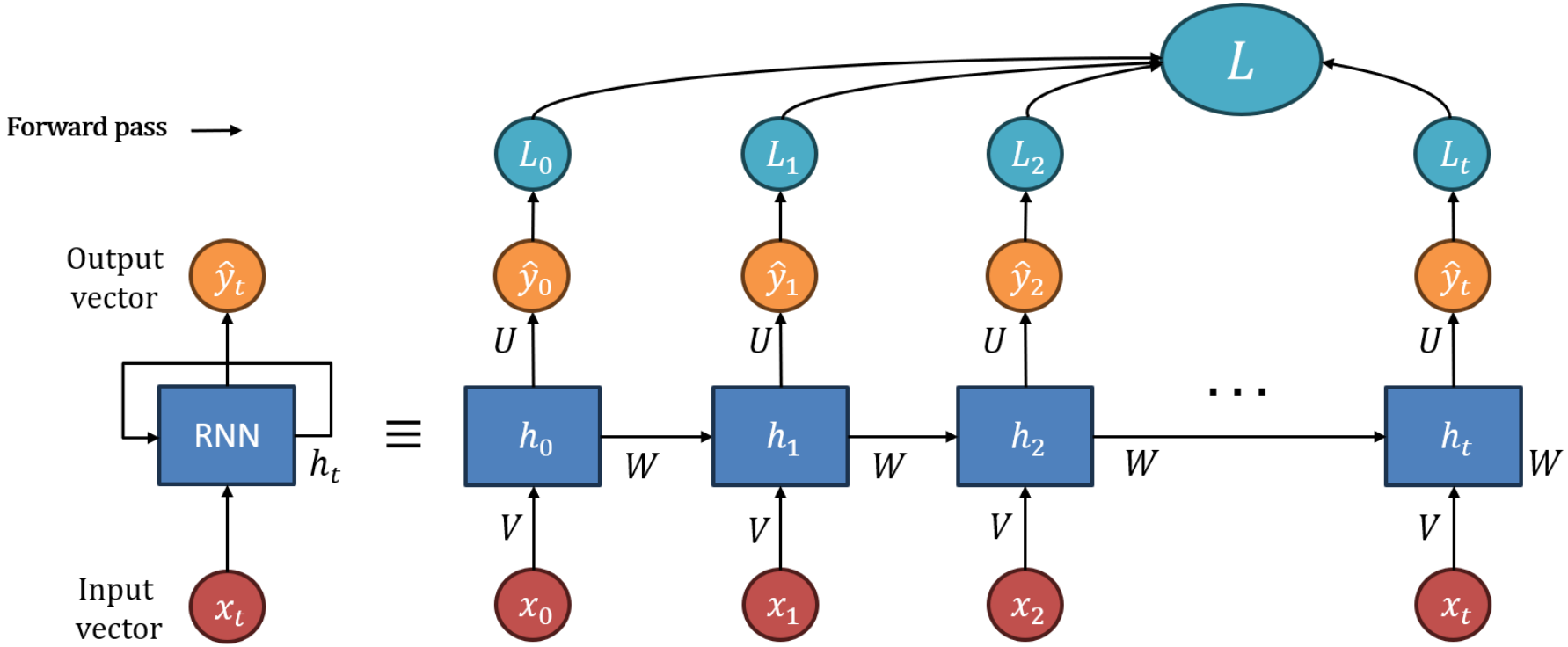
Standard RNN / Vanilla RNN Architecture

- Hidden states serve as the memory for RNN
- The forward pass of a vanilla RNN:
  - The same as that of an MLP with a single hidden layer
  - Except that activations arrive at the hidden layer from both the current external input and the hidden layer activations one step back in time.
  - Essentially copies of the RNN cell are made over time (unrolling/unfolding), with different inputs at different time steps. The weights are shared over time.



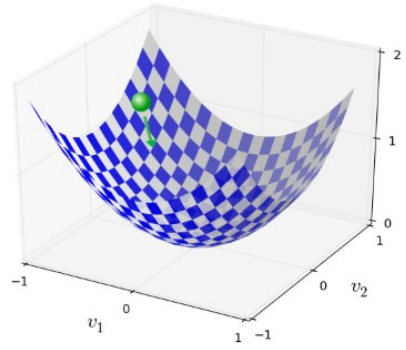
Training of Vanilla RNN

- RNN can be differentiated with respect to suitable objective functions, and thereby they could be trained with any gradient-descent based algorithm.
  - Treat the unrolled network as a one big feed forward network



Training of Vanilla RNN

- RNN can be differentiated with respect to suitable objective functions, and thereby they could be trained with any gradient-descent based algorithm.
  - Treat the unrolled network as a one big feed forward network
  - Compute gradients by back-propagation through time (BPTT)
  - Update shared weights

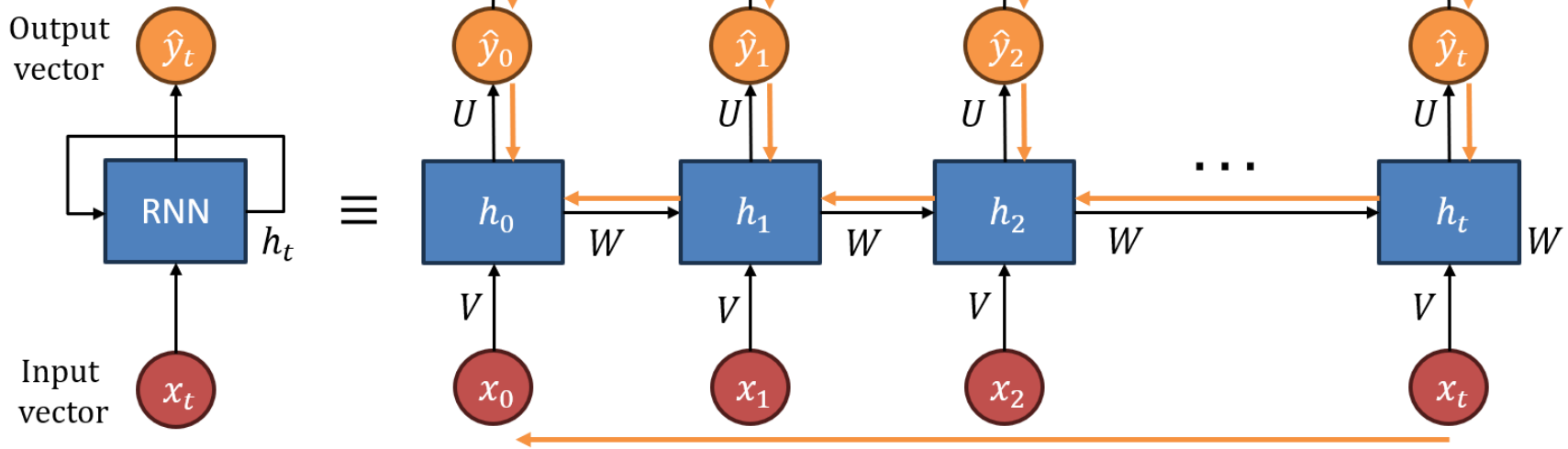


Forward pass →

Backward pass ←

Backward Pass:

$$\frac{\partial \mathbf{L}}{\partial U}, \frac{\partial \mathbf{L}}{\partial V}, \frac{\partial \mathbf{L}}{\partial W}, \frac{\partial \mathbf{L}}{\partial b_h}, \frac{\partial \mathbf{L}}{\partial b_y}$$



Variants of RNN

- RNNs are very effective for sequence tasks because they have "memory."
- They can read inputs (such as signal sample) one at a time, and remember some "contextual" information through the hidden layer activations that get passed from one time-step to the next.
- This allows two variants for sequence analysis:
  - Unidirectional (one-way) RNN: takes information from the past to process later inputs.
  - Bidirectional (two-way) RNN: can take context from both the past and the future (in many applications, output at time t may depend on the whole input sequence such as speech recognition where correct interpretation of the current sound may depend on the next few phonemes, potentially even the next few words)

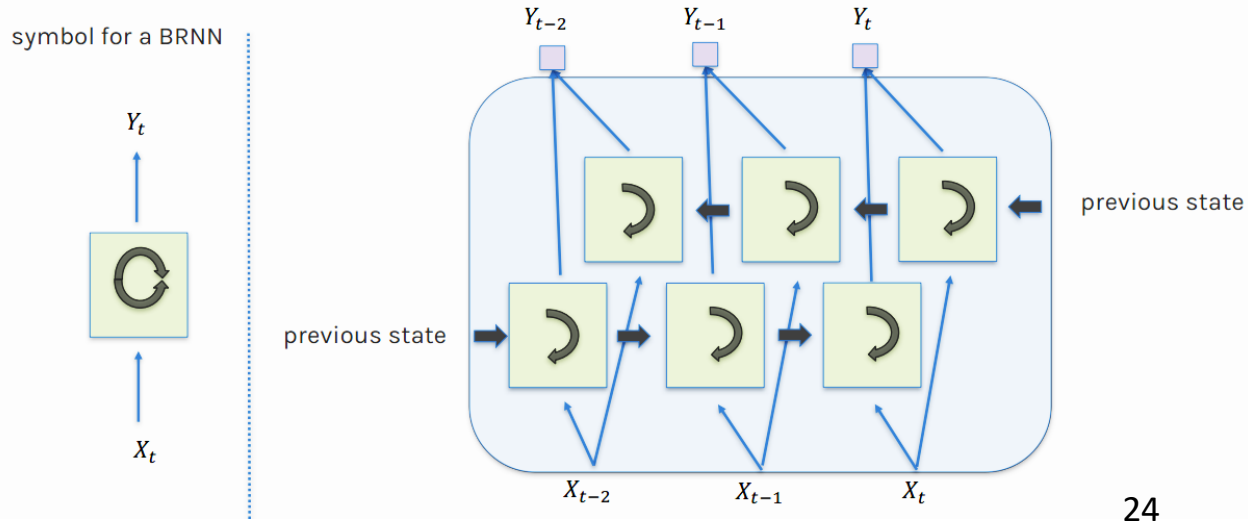
Bidirectional RNN

- A Bidirectional RNN is a recurrent neural network used primarily for natural language processing. Unlike standard RNN, the input flows in both directions, and it's capable of utilizing information from both sides, which makes it a powerful tool for modeling the sequential dependencies between words and phrases in both directions of the sequence.
- In some tasks, you have access to the full sequence at once (e.g., named entity recognition, sentiment analysis, where you're given the entire sentence upfront, and want to label each word).
- Bi-RNN consists of two RNNs: one processes the sequence forward, the other backward. Their outputs are combined at each timestep.
- For example, in the sentence "Apple is something that …", the word Apple might be about the apple as a fruit or about the company Apple. Unidirectional RNN can only use past context, so it may misinterpret words that require future context to predict.

"Apple is something that competitors simply cannot reproduce."

and

"Apple is something that I like to eat."



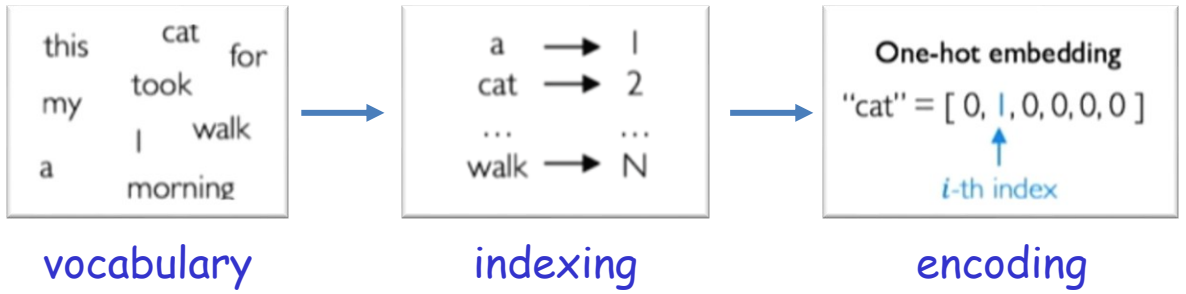
RNN – Encoding (for language)

Ex. NLP: How to represent language to RNN ?

- RNN can not interpret words/letters (needs numerical inputs to do the mathematical calculations) !

  Input X: "I took my cat for a walk this morning."

- One hot encoding (OHE)
  - Vocabulary: Create a vocabulary of the words in the sentence/document
  - Indexing: Sort the words in vocabulary (*e.g.* in ascending order A-Z and assign numerical indices)
  - Encoding: Transform to binary vectors (each word in the vocabulary is represented by a vector of size 'n', where 'n' is the total number of words in the vocabulary).



5.3. Gradient Vanishing/Exploding Problems

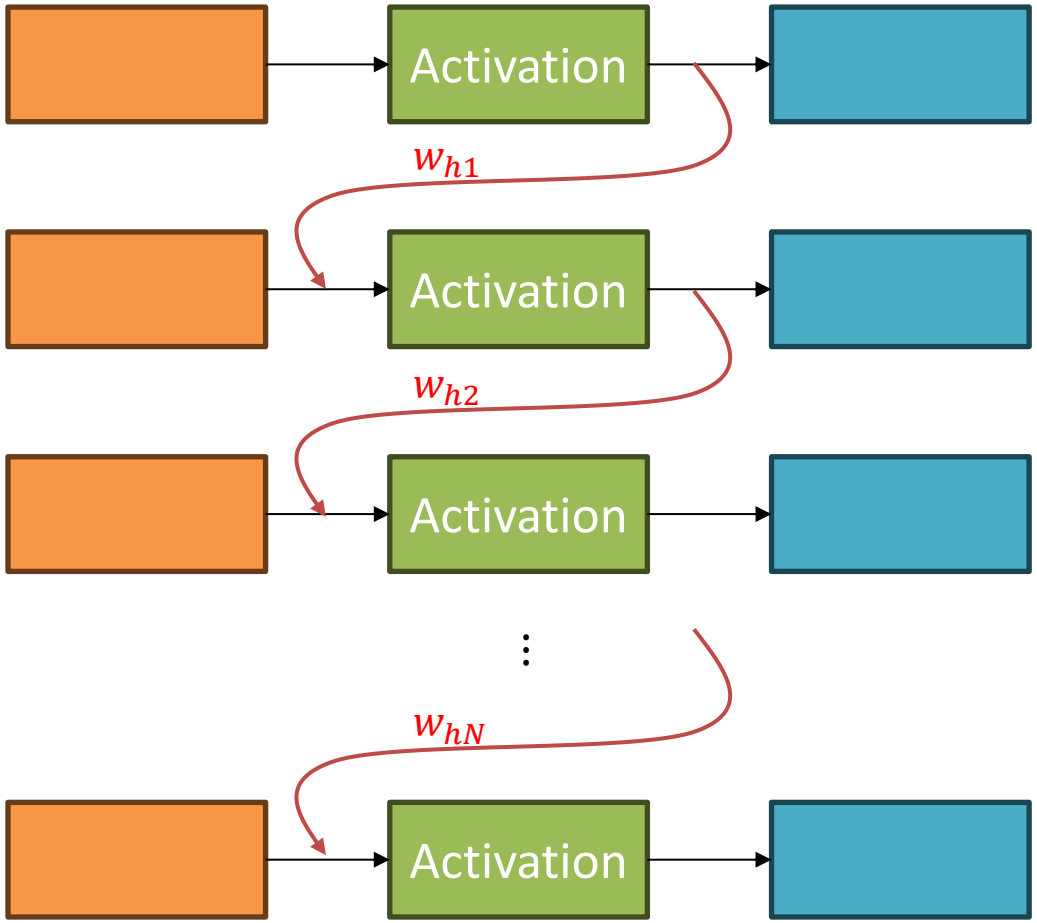
Issues with the Vanilla RNNs

- Gradient vanishing/exploding
  - In the same way a product of k real numbers can shrink to zero or explode to infinity, in RNNs, during the gradient back propagation phase, the gradient signal can end up being multiplied many times.

$$\frac{\partial \mathbf{L}}{\partial W} = \sum_{i=0}^{T} \frac{\partial \mathcal{L}_i}{\partial W} \propto \sum_{i=0}^{T} \left(\prod_{i=k+1}^{y} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W}$$

Vanishing gradient $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 < 1$

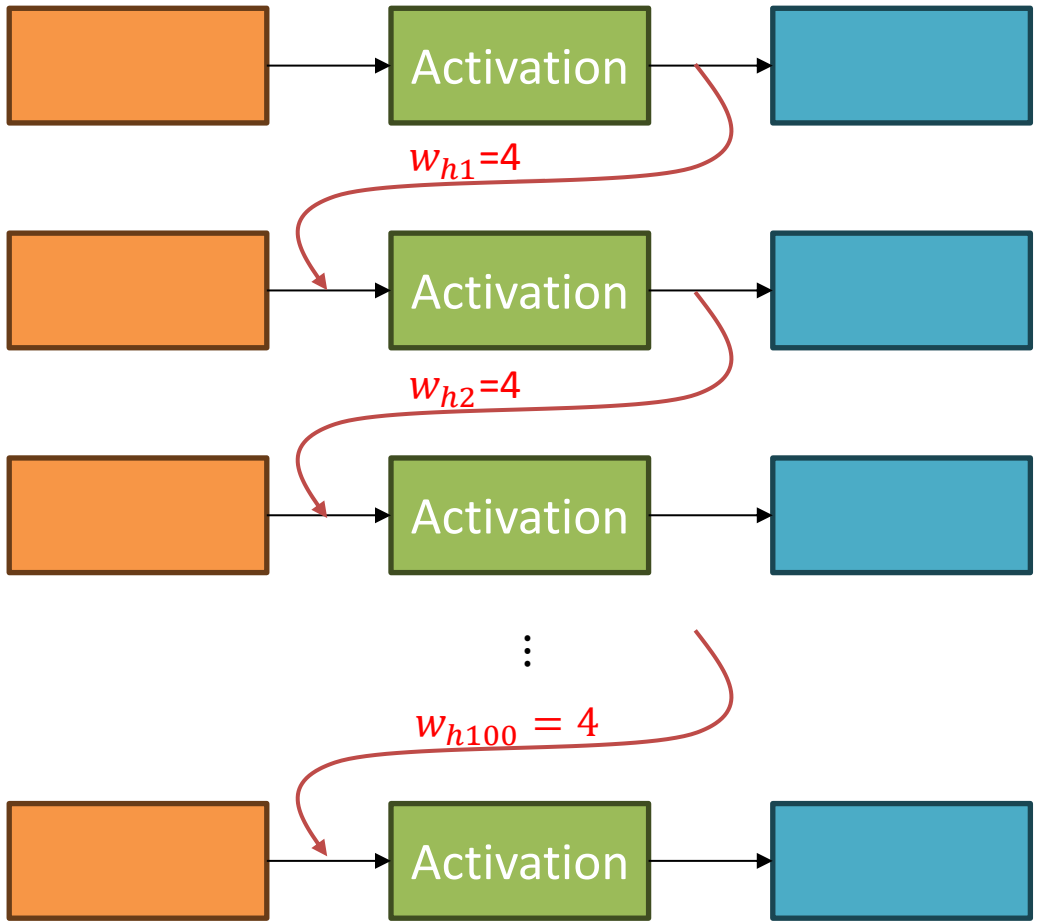
Exploding gradient $\left\| \frac{\partial h_i}{\partial h_{i-1}} \right\|_2 > 1$



Issues with the Vanilla RNNs

- Gradient exploding
  - If the gradients are large → exploding gradients, learning diverges
  - Imagine the weight is equal to 4
  - Let's say we have 100 sequential data points
  - Then we would unroll the network 100 times
  - We'll multiply input by a huge number

$$Input \times 4 \times 4 \times 4 \times 4 \ldots \times 4 = Input \times 4^{100}$$

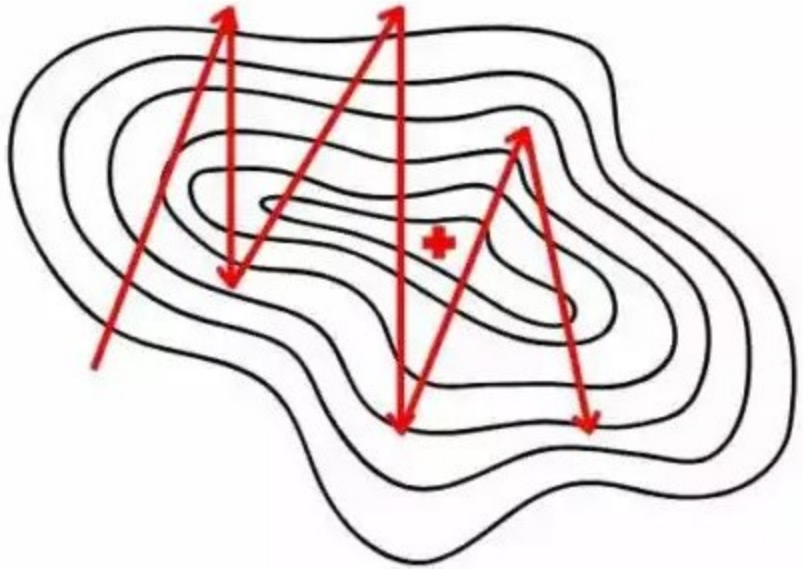


Issues with the Vanilla RNNs

- Gradient exploding
  - If the gradients are large → exploding gradients, learning diverges
  - Imagine the weight is equal to 4
  - Let's say we have 100 sequential data points
  - Then we would unroll the network 100 times
  - We'll multiply input by a huge number

$$Input \times 4 \times 4 \times 4 \times 4 \ldots \times 4 = Input \times 4^{100}$$

  - This huge number causes the gradient in the gradient descent calculation to explode

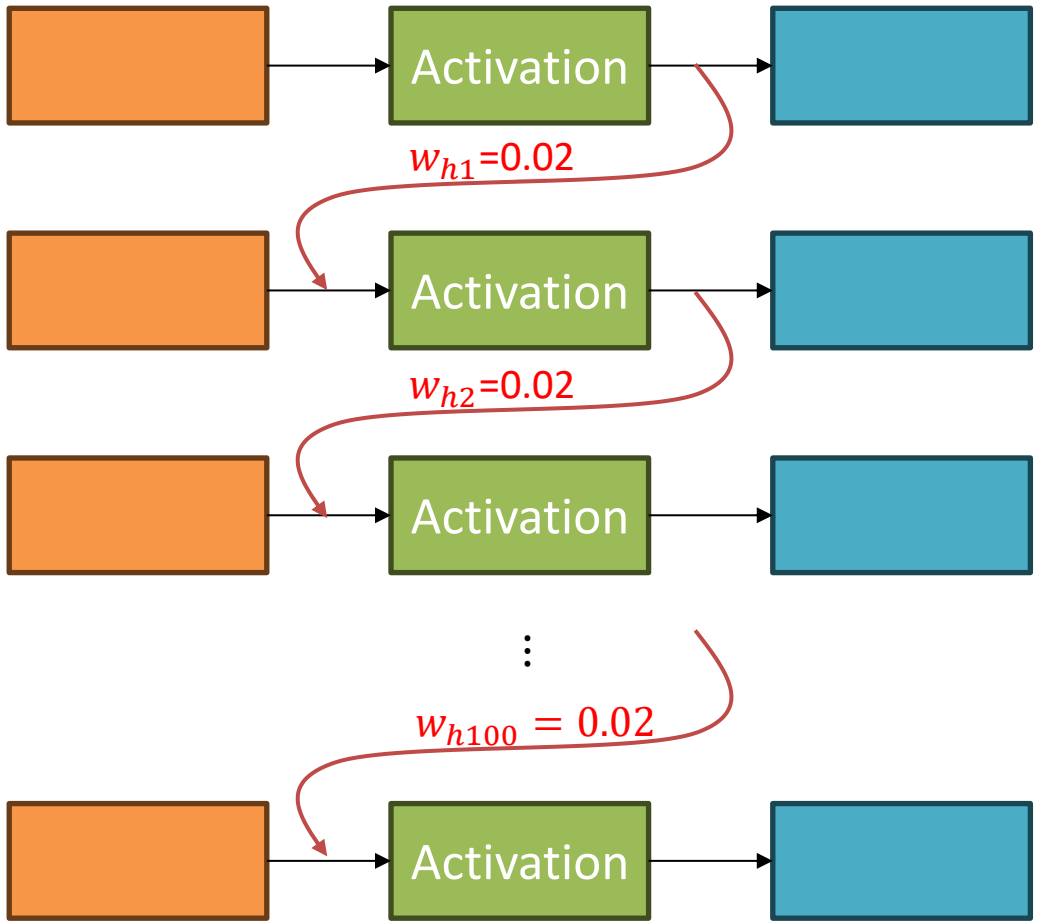


Solution: clip the gradients to a certain max value or limit weights to less than 1.

Issues with the Vanilla RNNs

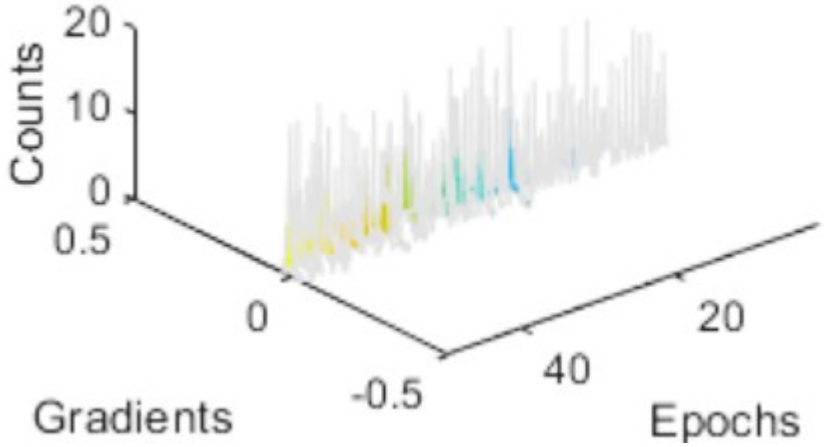
- Gradient vanishing
  - If the gradients are small → vanishing gradients, learning very slow or stops
  - Imagine the weight is equal to 0.02
  - Let's say we have 100 sequential data points, then we would unroll the network 100 times
  - We'll end up with multiplying input by a very small number

$$Input \times 0.02 \times 0.02 \times 0.02 \times 0.02 \ldots \times 0.02 = Input \times (0.02)^{100}$$



Issues with the Vanilla RNNs

- Gradient vanishing
  - If the gradients are small → vanishing gradients, learning very slow or stops
  - Imagine the weight is equal to 0.02
  - Let's say we have 100 sequential data points, then we would unroll the network 100 times
  - We'll end up with multiplying input by a very small number



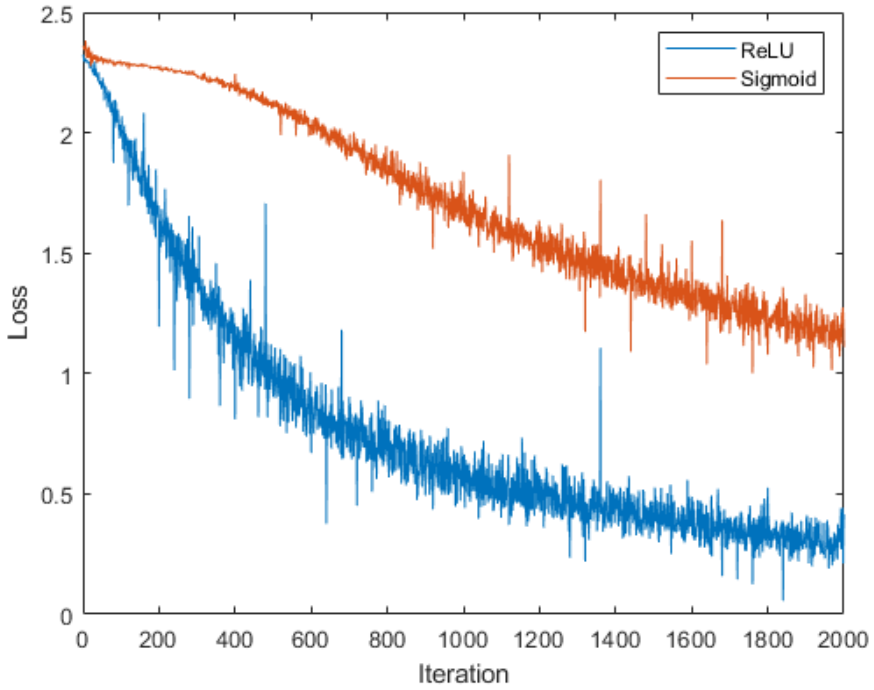
$$Input \times 0.02 \times 0.02 \times 0.02 \times 0.02 \ldots \times 0.02 = Input \times (0.02)^{100}$$

  - This small number causes the gradient in the gradient descent calculation to vanish and biases parameters to capture only short-term dependencies.
  - We will end up hitting max iteration before finding optimal value

Issues with the Vanilla RNNs

- Gradient vanishing
  - If the gradients are small → vanishing gradients, learning very slow or stops
  - Imagine the weight is equal to 0.02
  - Let's say we have 100 sequential data points, then we would unroll the network 100 times
  - We'll end up with multiplying input by a very small number

$$Input \times 0.02 \times 0.02 \times 0.02 \times 0.02 \ldots \times 0.02 = Input \times (0.02)^{100}$$



  - This small number causes the gradient in the gradient descent calculation to vanish and biases parameters to capture only short-term dependencies.
  - We will end up hitting max iteration before finding optimal value

Solutions: use proper activation, or an even better solution by introducing gated cells to selectively add/remove information within each recurrent unit (LSTM/GRU, etc.)

5.4. Long-Short Term Memory (LSTM)

Long-Short Term Memory (LSTM)

- LSTMs are RNNs capable of learning long-term dependencies.
- LSTM cells are able to track information throughout many timesteps
- A memory cell using logistic and linear units with multiplicative interactions
- Uses gates to control the flow of information:
  - Relevant information gets into the cell whenever its **input gate** is on (write).
  - Irrelevant information is thrown away from the cell whenever its **forget gate** is off (reset).
  - Information can be read from the cell by turning on its **output gate** (read).

Pointwise operation

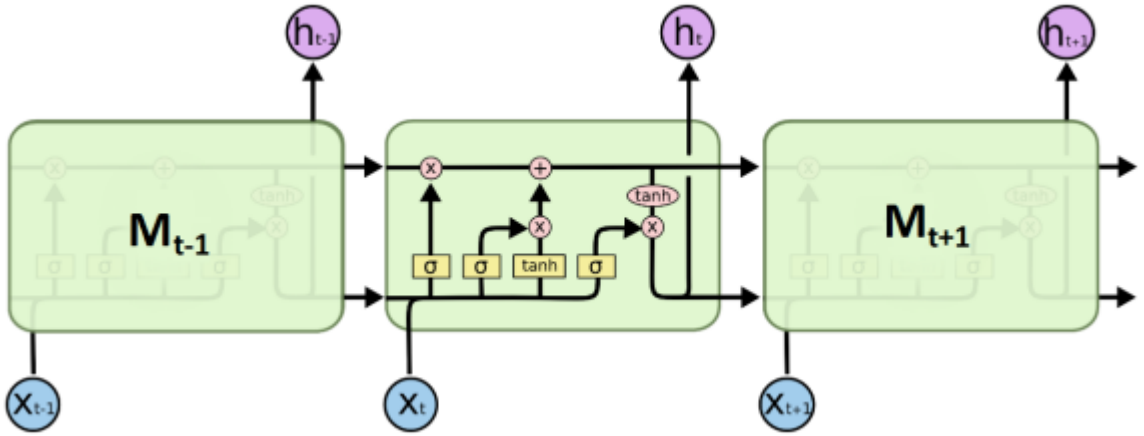


Neural Network Layer
(sigmoid and tanh)



how much
info to pass

what content to store, with
positive and negative influence



LSTM - Memory Cell

Purpose of memory cells is to remember things for a long time.

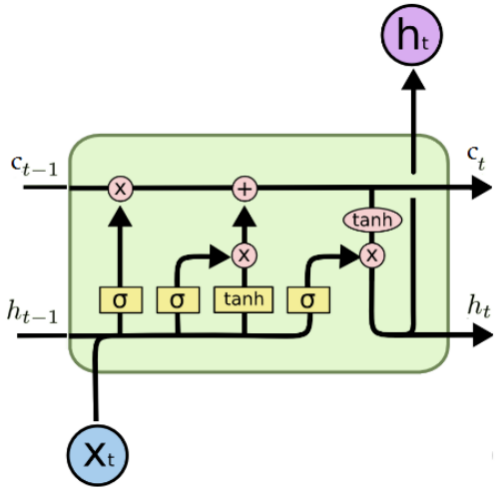
- Information can flow along the memory cell unchanged.
- Information can be removed or written to the memory cell, regulated by gates.
- Gates are a way to optionally let information through.
  - A sigmoid layer outputs number between 0 and 1, deciding how much of each component should be let through.
  - A pointwise multiplication operation applies the decision.
- Information gets into the cell when its input gate is on.
- Information is read from the cell when the output gate is on.
- Information is thrown away when the forget gate is off (values near 1 = keep, values near 0 = forget)

Sigmoid (σ)

- Range: **0 → 1**
- Used in **gates** (input, forget, output)
- Acts like a **soft switch**:
  - 0 = block, 1 = pass, in-between = partial flow

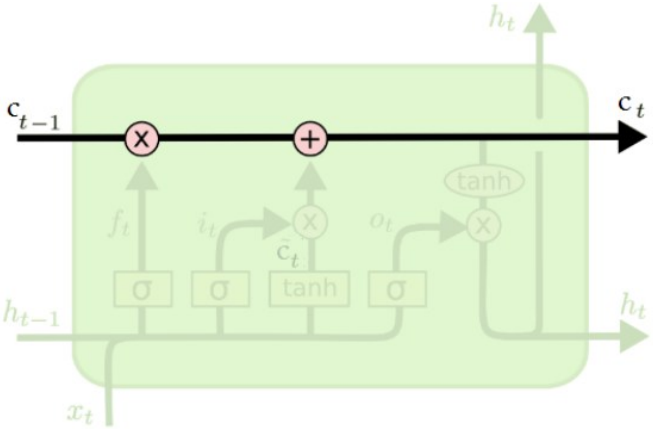
tanh

- Range: **-1 → 1**
- Used for **cell updates & output squashing**
- Keeps values bounded
- Allows **positive & negative memory** (activate vs. inhibit)



LSTM - Memory Cell

- Cell state (long-term memory)
- There are no weights and biases that can modify it directly (long-term memory flows without causing the gradients to explode or vanish)

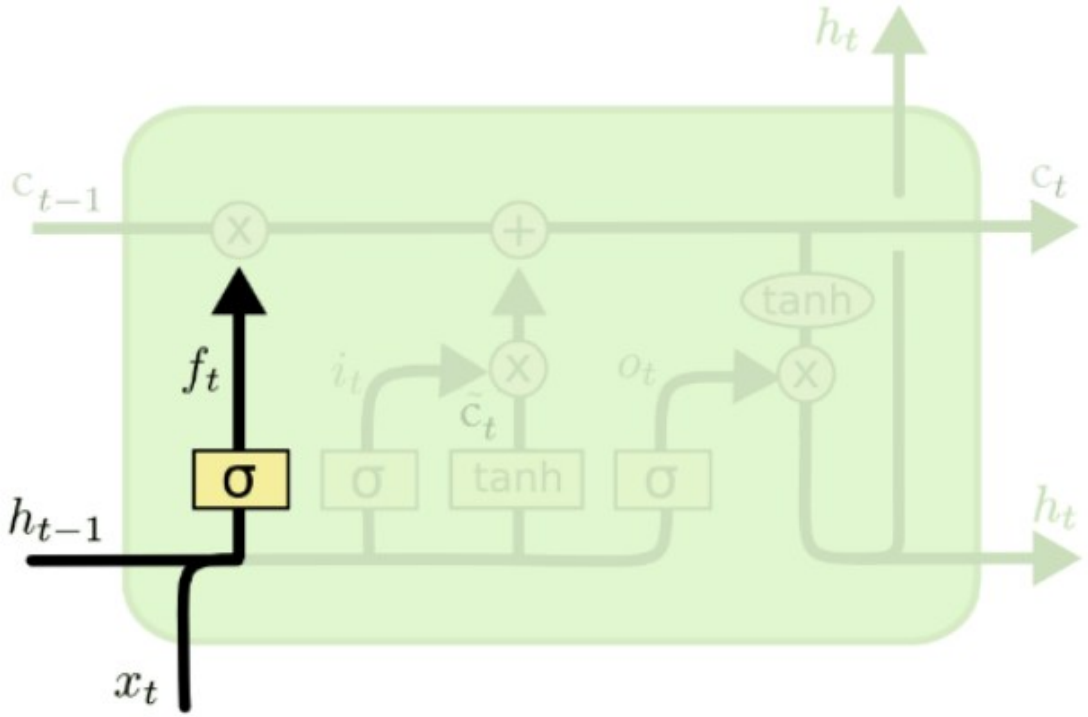


- Hidden states (short-term memories)

LSTM - Memory Cell

- A sigmoid layer, "forget gate", controls what is kept vs. forgotten from previous cell state (determines what percentage of long-term memory is remembered).

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

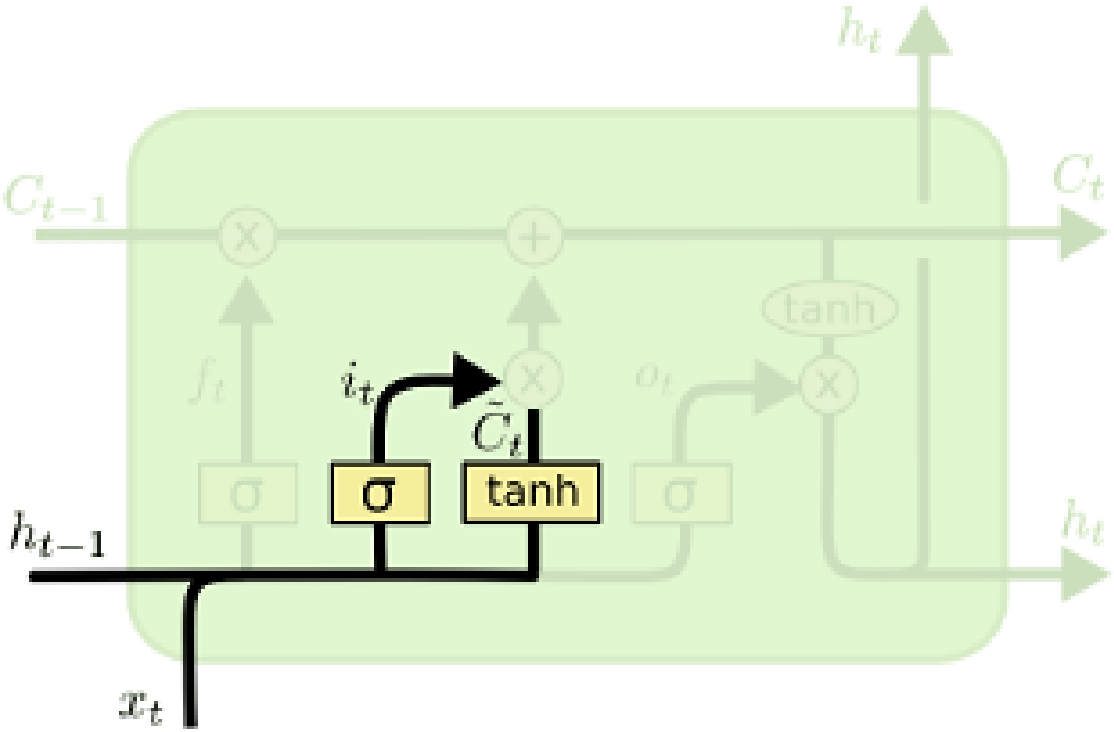


LSTM - Memory Cell

- A sigmoid layer, "input gate", controls what parts of new cell content are written to cell.

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

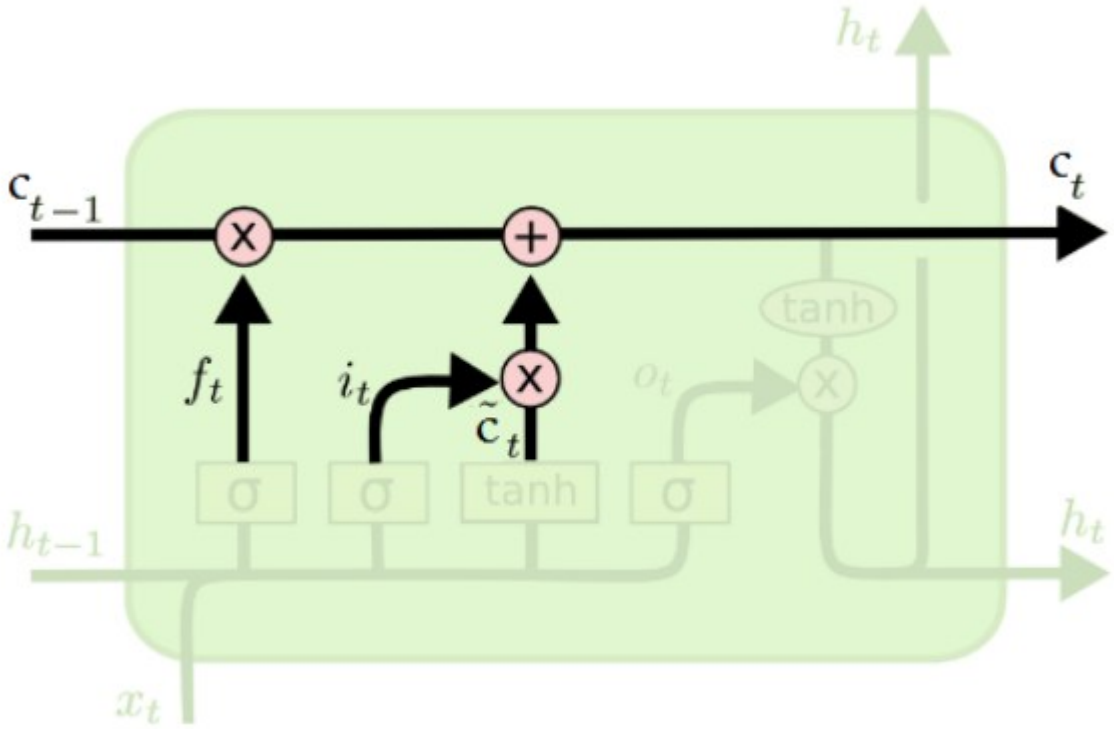
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$



LSTM - Memory Cell

- A Tanh layer creates a vector of new candidate values to write to the memory cell
- The previous steps decided which values of the memory cell to reset and overwrite.
- Now the LSTM applies the decisions to the memory cell, "memory update".
  (erases/forget some content from the cell state, and write/input some new cell content)

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

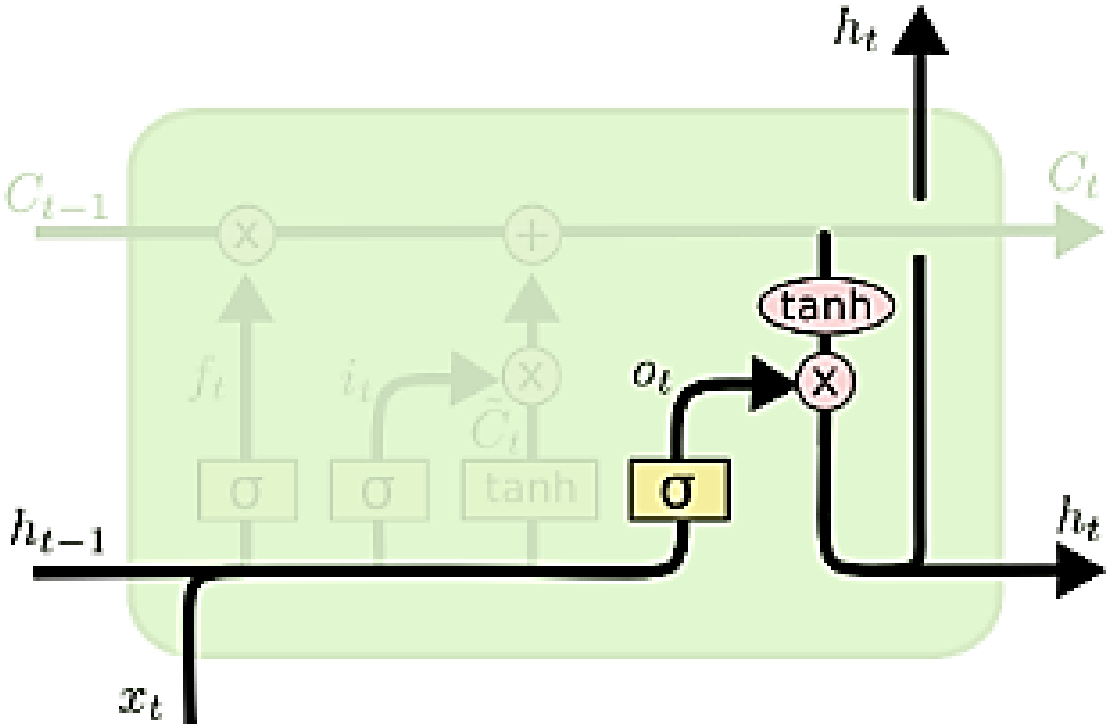


LSTM - Memory Cell

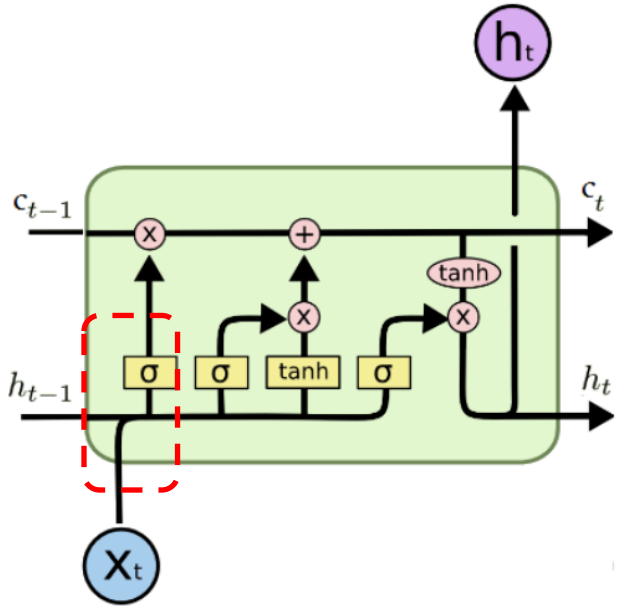
- A sigmoid layer, "output gate", controls what parts of cell are output to hidden state.
- The memory cell goes through Tanh and is multiplied by the output gate ("output update").

$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$



Forget Gate



Cell State

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_{t-1}] + b_f)$$

Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_f)$$

$$x_t = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \qquad h_{t-1} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \qquad C_{t-1} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}$$

$$W_f = \begin{bmatrix} 1 & -1 & 0.5 & 0.5 & 0 \\ 0 & 1 & -0.5 & 0.5 & 0 \end{bmatrix} \qquad b_f = \begin{bmatrix} -100 \\ -100 \end{bmatrix}$$

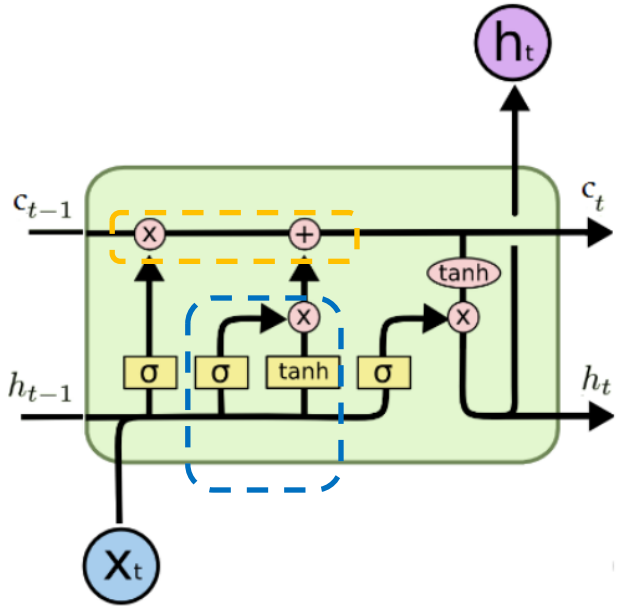
$$[h_{t-1}; x_t] = \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \\ 1 \end{bmatrix}$$

$$f_t = \sigma\left(\begin{bmatrix} 1 \times 1 + (-1) \times 1 + 0.5 \times 1 + (-1) \times 1 + 0 \times 1 \\ 0 \times 1 + 1 \times 1 + (-0.5) \times 1 + 0.5 \times (-1) + 0 \times 1 \end{bmatrix} + \begin{bmatrix} -100 \\ -100 \end{bmatrix}\right)$$

$$f_t = \sigma\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -100 \\ -100 \end{bmatrix}\right)$$

$$f_t \approx \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Input Gate



Cell State

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_{t-1}] + b_f)$$

Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_f)$$

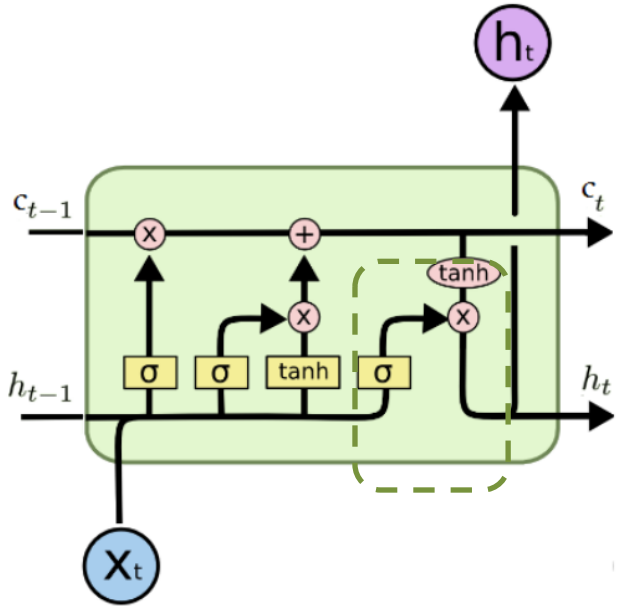
For example: Input gate estimation is: $i_t = \begin{bmatrix}1\\1\end{bmatrix}, C_t = \begin{bmatrix}1\\1\end{bmatrix}$

$$\tilde{C}_t = tanh\left(\begin{bmatrix}1+0+1+1+1\\0+1-1-1+1\end{bmatrix} + \begin{bmatrix}-100\\-100\end{bmatrix}\right)$$

$$\tilde{C}_t = tanh\left(\begin{bmatrix}1\\1\end{bmatrix} + \begin{bmatrix}-100\\-100\end{bmatrix}\right)$$

$$\tilde{C}_t \approx \begin{bmatrix}-1\\-1\end{bmatrix}$$

Output Gate



Cell State

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_{t-1}] + b_f)$$

Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_f)$$

- It is important to notice that hidden state used in the next step is obtained using the output gate layer which is usually the function that we optimize.

$$W_O = \begin{bmatrix} 1 & 1 & 1 & -1 & 1 \\ -1 & 1 & 0.5 & 0.5 & 1 \end{bmatrix} \qquad b_O = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$O_t = \sigma\left(\begin{bmatrix} 1\times 1 + 1\times 1 + 1\times 1 + (-1)\times(-1) + 1\times 1 \\ (-1)\times 1 + 1\times 1 + 0.5\times 1 + 0.5\times(-1) + 1\times 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}\right)$$

$$O_t \approx \sigma\left(\begin{bmatrix} 5 \\ 1 \end{bmatrix}\right) \approx \begin{bmatrix} 0.993 \\ 0.731 \end{bmatrix}$$

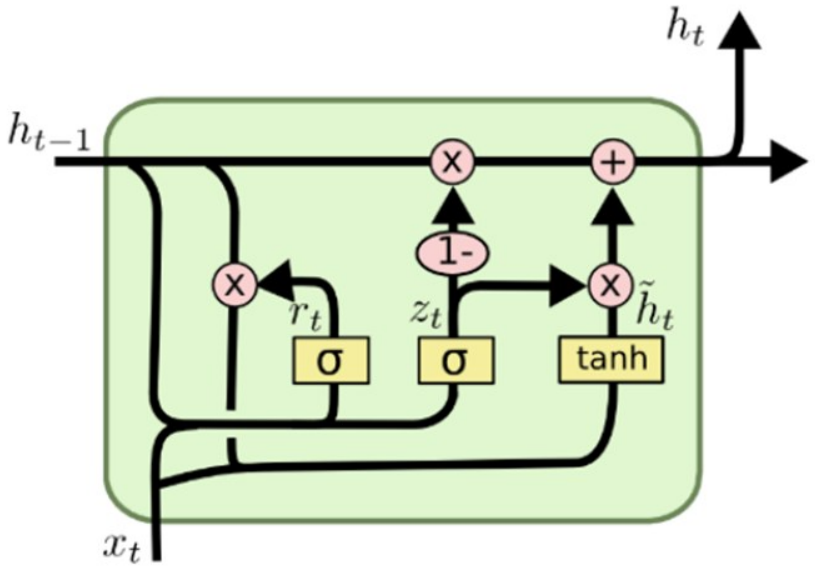
$$h_t \approx \begin{bmatrix} 0.993 \\ 0.731 \end{bmatrix} * \begin{bmatrix} 0.761 \\ 0.761 \end{bmatrix} = \begin{bmatrix} 0.756 \\ 0.556 \end{bmatrix}$$

5.5. Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU)

- A new type of RNN cell (Gated Feedback Recurrent Neural Networks)
- Very similar to LSTM but merges the memory cell and the hidden state.
- It combines the **forget** and **input** gates into a single **update** gate.
- Computationally more efficient (less parameters, less complex structure)

(2 gates instead of 3 ≡ fewer weights).



Summary

- Advantages:
  - RNN: Simplicity, effective for basic sequential tasks.
  - LSTM: Powerful for complex sequences, handles long-term dependencies well.
  - GRU: Faster training, simpler architecture, often performs as well as LSTMs.

- Challenges:
  - RNN: Vanishing gradient problem, limited to short-term dependencies.
  - LSTM: High computational cost, complexity in tuning.
  - GRU: May require empirical testing to determine if performance is on par with LSTMs for specific tasks.

5.6. LSTM for Time Series Prediction

Univariate LSTM Models

- LSTMs can be used to model univariate time series forecasting problems
  1. Data Preparation:
     - the sequence of observations must be transformed into multiple examples from which the model can learn (format suitable for supervised learning).
     - the model will expect the input component of training data to have the dimensions or shape: [samples, timesteps, features].

```
# univariate data preparation
from numpy import array
# split a univariate sequence into samples
def split_sequence(sequence, n_steps):
    X, y = list(), list()
    for i in range(len(sequence)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the sequence
        if end_ix > len(sequence)-1:
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)

# define input sequence
raw_seq = [10, 20, 30, 40, 50, 60, 70, 80, 90]
# choose a number of time steps
n_steps = 3
# split into samples
X, y = split_sequence(raw_seq, n_steps)
# summarize the data
for i in range(len(X)):
    print(X[i], y[i])
```

```
[10 20 30] 40
[20 30 40] 50
[30 40 50] 60
[40 50 60] 70
[50 60 70] 80
[60 70 80] 90
```

```
# reshape from [samples, timesteps] into [samples, timesteps, features]
n_features = 1 # univariate
X = X.reshape((X.shape[0], X.shape[1], n_features))
for i in range(len(X)):
    print(X[i], y[i])
```

```
[[10]
 [20]
 [30]] 40
[[20]
 [30]
 [40]] 50
[[30]
 [40]
 [50]] 60
[[40]
 [50]
 [60]] 70
[[50]
 [60]
 [70]] 80
[[60]
 [70]
 [80]] 90
```

Univariate LSTM Models – Vanilla LSTM

2. Vanilla LSTM model
   - A Vanilla LSTM is an LSTM model that has a single hidden layer of LSTM units, and an output layer used to make a prediction. Key to LSTMs is that they offer native support for sequences.
   - Unlike a CNN that reads across the entire input vector, the LSTM model reads one time step of the sequence at a time and builds up an internal state representation that can be used as a learned context for making a prediction.

```
# define model
from keras.models import Sequential
from keras.layers import LSTM
from keras.layers import Dense
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(n_steps, n_features))) #50 LSTM units in the hidden layer
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
print(model.summary())
```

```
Model: "sequential_4"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_2 (LSTM)               (None, 50)                10400

 dense_2 (Dense)             (None, 1)                 51

=================================================================
Total params: 10,451
Trainable params: 10,451
Non-trainable params: 0
_________________________________________________________________
None
```

```
# fit the model on the training dataset
model.fit(X, y, epochs=200, verbose=0)
```

```
# Predict the next value in the sequence
x_input = array([70, 80, 90])
# The model expects the input shape to be
# three-dimensional with [samples, timesteps, features]
x_input = x_input.reshape((1, n_steps, n_features))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

```
[[103.16055]]
```

Univariate LSTM Models – Stacked LSTM

2. Stacked LSTM model
   - Multiple hidden LSTM layers can be stacked one on top of another (Stacked LSTM model).
   - LSTM layer requires a 3D input and by default will produce a 2D output.
   - We address this by having LSTM output a value for each time step in the input data by setting the return sequences=True argument on the layer.
   - This allows us to have 3D output from hidden LSTM layer as input to the next.

```
# define model
model = Sequential()
model.add(LSTM(50, activation='relu', return_sequences=True, input_shape=(n_steps,
n_features)))
model.add(LSTM(50, activation='relu'))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
print(model.summary())
```

```
Model: "sequential_6"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 lstm_5 (LSTM)                (None, 3, 50)             10400

 lstm_6 (LSTM)                (None, 50)                20200

 dense_4 (Dense)              (None, 1)                 51

=================================================================
Total params: 30,651
Trainable params: 30,651
Non-trainable params: 0
_________________________________________________________________
None
```

```
# fit the model
model.fit(X, y, epochs=200, verbose=0)
# demonstrate prediction
x_input = array([70, 80, 90])
x_input = x_input.reshape((1, n_steps, n_features))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

```
[[104.56153]]
```

Univariate LSTM Models – Bidirectional LSTM

2. Bidirectional LSTM model
   - On some sequence prediction problems, it can be beneficial to allow the LSTM model to learn the input sequence both forward and backward and concatenate both interpretations.
   - A Bidirectional LSTM for univariate time series forecasting can be built by wrapping the first hidden layer in a wrapper layer called Bidirectional

```
# define model
from keras.layers import Dense, LSTM, Bidirectional
model = Sequential()
model.add(Bidirectional(LSTM(50, activation='relu'), input_shape=(n_steps, n_features)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
print(model.summary())
```

```
Model: "sequential_8"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 bidirectional (Bidirectiona  (None, 100)              20800
 l)

 dense_5 (Dense)             (None, 1)                 101

=================================================================
Total params: 20,901
Trainable params: 20,901
Non-trainable params: 0
_________________________________________________________________
None
```

```
# fit model
model.fit(X, y, epochs=200, verbose=0)
# demonstrate prediction
x_input = array([70, 80, 90])
x_input = x_input.reshape((1, n_steps, n_features))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

```
[[101.70904]]
```

Multivariate LSTM Models

Multivariate time series data means data where there is more than one observation for each time step. There are two main models:

- Multiple Input Series.
- Multiple Parallel Series.

```
(7, 3, 2) (7,)

[[10 15]
 [20 25]
 [30 35]] 65
[[20 25]
 [30 35]
 [40 45]] 85
[[30 35]
 [40 45]
 [50 55]] 105
[[40 45]
 [50 55]
 [60 65]] 125
[[50 55]
 [60 65]
 [70 75]] 145
[[60 65]
 [70 75]
 [80 85]] 165
[[70 75]
 [80 85]
 [90 95]] 185
```

```python
# split a multivariate sequence into samples
def split_sequences(sequences, n_steps):
    X, y = list(), list()
    for i in range(len(sequences)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the dataset
        if end_ix > len(sequences):
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequences[i:end_ix, :-1], sequences[end_ix-1, -1]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)
# define input sequence
in_seq1 = array([10, 20, 30, 40, 50, 60, 70, 80, 90])
in_seq2 = array([15, 25, 35, 45, 55, 65, 75, 85, 95])
out_seq = array([in_seq1[i]+in_seq2[i] for i in range(len(in_seq1))])
# convert to [rows, columns] structure
in_seq1 = in_seq1.reshape((len(in_seq1), 1))
in_seq2 = in_seq2.reshape((len(in_seq2), 1))
out_seq = out_seq.reshape((len(out_seq), 1))
# horizontally stack columns
dataset = hstack((in_seq1, in_seq2, out_seq))
# choose a number of time steps
n_steps = 3
# convert into input/output
X, y = split_sequences(dataset, n_steps)
# the dataset knows the number of features, e.g. 2
n_features = X.shape[2]
# define model
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(n_steps, n_features)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
# fit model
model.fit(X, y, epochs=200, verbose=0)
# demonstrate prediction
x_input = array([[80, 85], [90, 95], [100, 105]])
x_input = x_input.reshape((1, n_steps, n_features))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

```
[[205.66463]]
```

```
(6, 3, 3) (6, 3)

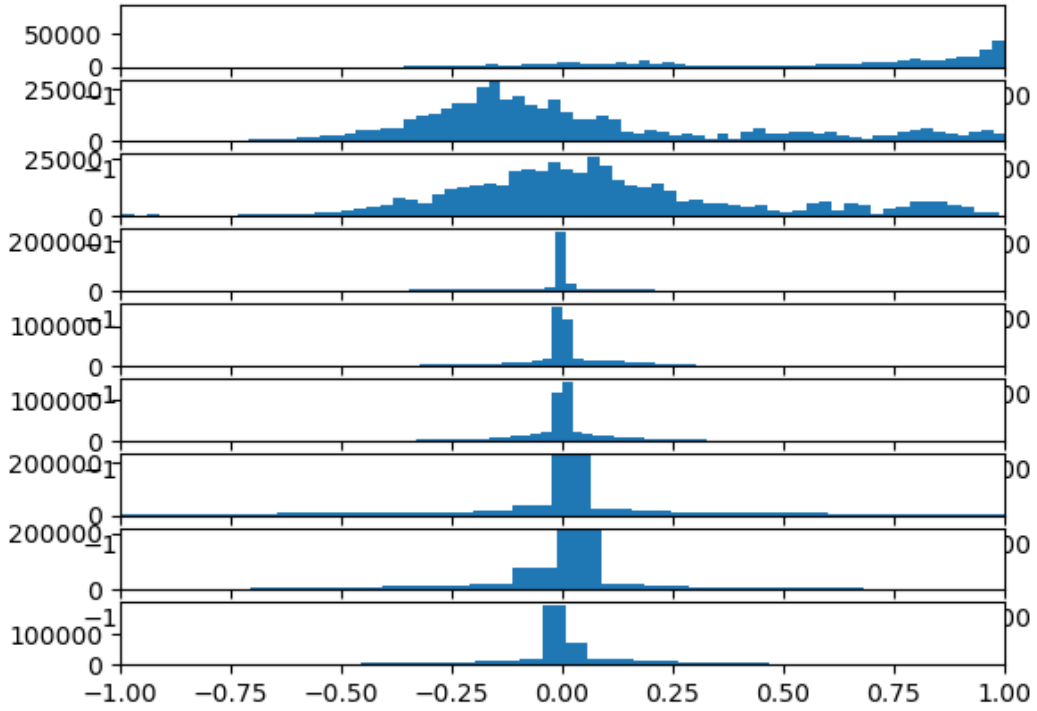
[[10 15 25]
 [20 25 45]
 [30 35 65]] [40 45 85]
[[20 25 45]
 [30 35 65]
 [40 45 85]] [ 50 55 105]
[[ 30  35  65]
 [ 40  45  85]
 [ 50  55 105]] [ 60 65 125]
[[ 40  45  85]
 [ 50  55 105]
 [ 60  65 125]] [ 70 75 145]
[[ 50  55 105]
 [ 60  65 125]
 [ 70  75 145]] [ 80 85 165]
[[ 60  65 125]
 [ 70  75 145]
 [ 80  85 165]] [ 90 95 185]
```

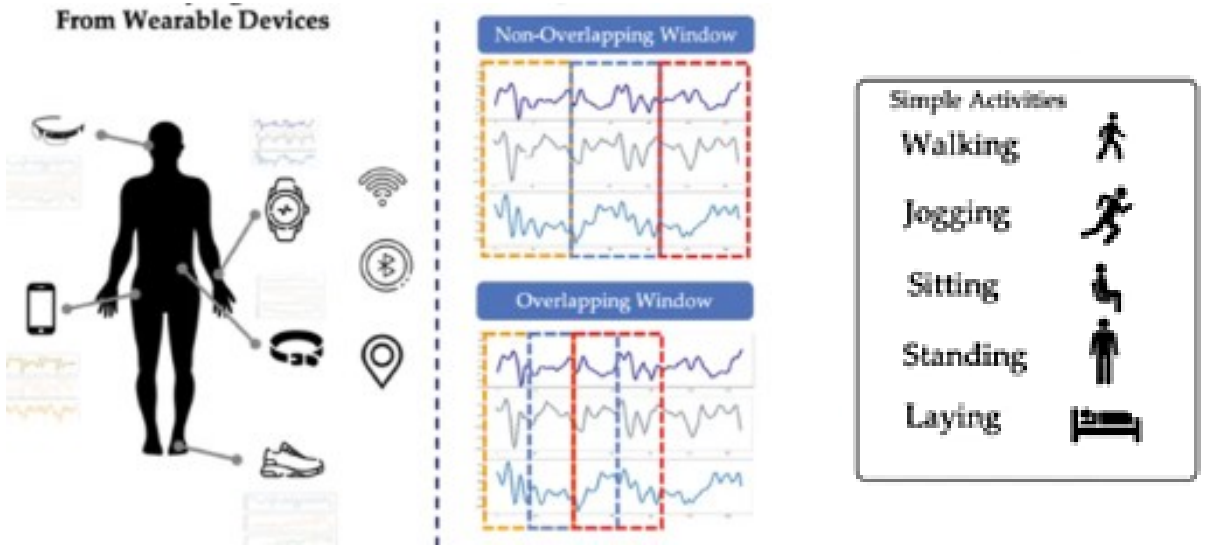
```python
# split a multivariate sequence into samples
def split_sequences(sequences, n_steps):
    X, y = list(), list()
    for i in range(len(sequences)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the dataset
        if end_ix > len(sequences)-1:
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequences[i:end_ix, :], sequences[end_ix, :]
        X.append(seq_x)
        y.append(seq_y)
    return array(X), array(y)
# define input sequence
in_seq1 = array([10, 20, 30, 40, 50, 60, 70, 80, 90])
in_seq2 = array([15, 25, 35, 45, 55, 65, 75, 85, 95])
out_seq = array([in_seq1[i]+in_seq2[i] for i in range(len(in_seq1))])
# convert to [rows, columns] structure
in_seq1 = in_seq1.reshape((len(in_seq1), 1))
in_seq2 = in_seq2.reshape((len(in_seq2), 1))
out_seq = out_seq.reshape((len(out_seq), 1))
# horizontally stack columns
dataset = hstack((in_seq1, in_seq2, out_seq))
# choose a number of time steps
n_steps = 3
# convert into input/output
X, y = split_sequences(dataset, n_steps)
# the dataset knows the number of features, e.g. 2
n_features = X.shape[2]
# define model
model = Sequential()
model.add(LSTM(100, activation='relu', return_sequences=True, input_shape=(n_steps,
    n_features)))
model.add(LSTM(100, activation='relu'))
model.add(Dense(n_features))
model.compile(optimizer='adam', loss='mse')
# fit model
model.fit(X, y, epochs=400, verbose=0)
# demonstrate prediction
x_input = array([[70,75,145], [80,85,165], [90,95,185]])
x_input = x_input.reshape((1, n_steps, n_features))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

```
[[ 99.70584 106.01055 206.22656]]
```

UCI Example:

- A UCI dataset of human action recognition (HAR) including walking, walking upstairs, walking downstairs, sitting, standing, laying.
- The movement data recorded was the x, y, and z accelerometer data (linear acceleration) and gyroscopic data (angular velocity) from the smart phone.
- Each subject performed the sequence of activities twice, once with the device on their left-hand-side and once with the device on their right-hand side.



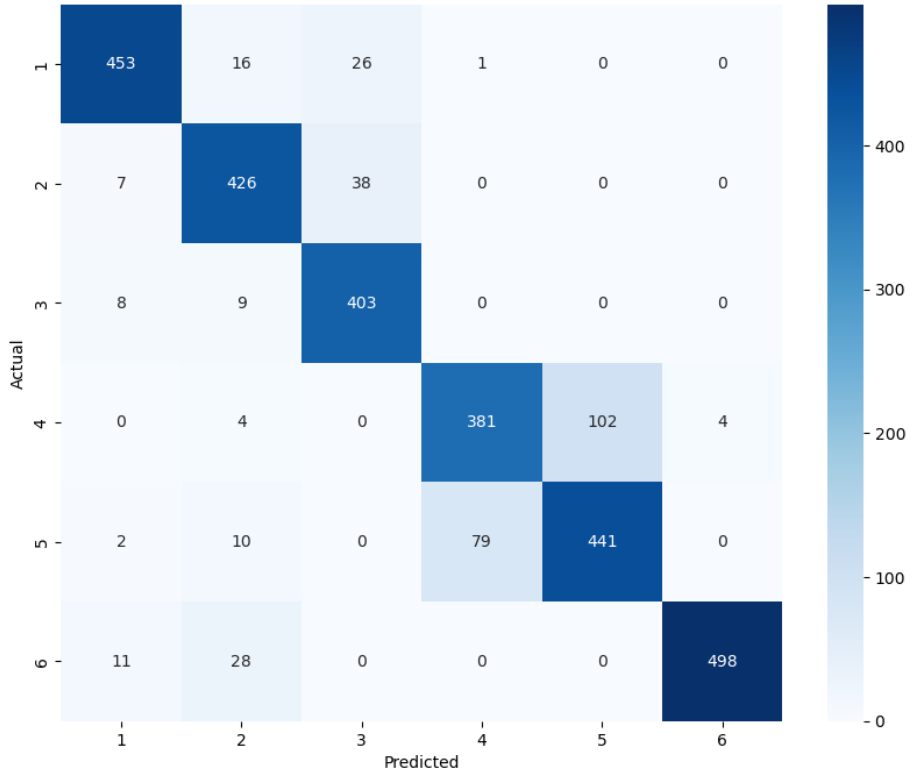


UCI Example:

- Building the models:
  - LSTM model
  - CNN model

LSTM:

1) first layer: a single LSTM hidden layer
2) followed by a dropout layer (to reduce overfitting)
3) finally, a dense fully connected layer
4) at last we have the softmax layer to represent 6 classes




```
n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
model = Sequential()
model.add(LSTM(100, input_shape=(n_timesteps,n_features)))
model.add(Dropout(0.5))
model.add(Dense(100, activation='relu'))
model.add(Dense(n_outputs, activation='softmax'))
plot_model(model, show_shapes=True, show_layer_names=True)
```

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

**** Classification Report ****

	precision	recall	f1-score
1	0.83	0.91	0.87
2	0.87	0.78	0.82
3	0.89	0.97	0.92
4	0.79	0.84	0.82
5	0.85	0.79	0.82
6	1.00	0.95	0.97
accuracy			0.87
macro avg	0.87	0.87	0.87
weighted avg	0.87	0.87	0.87

Trained a LSTM for 10 epochs

UCI Example:

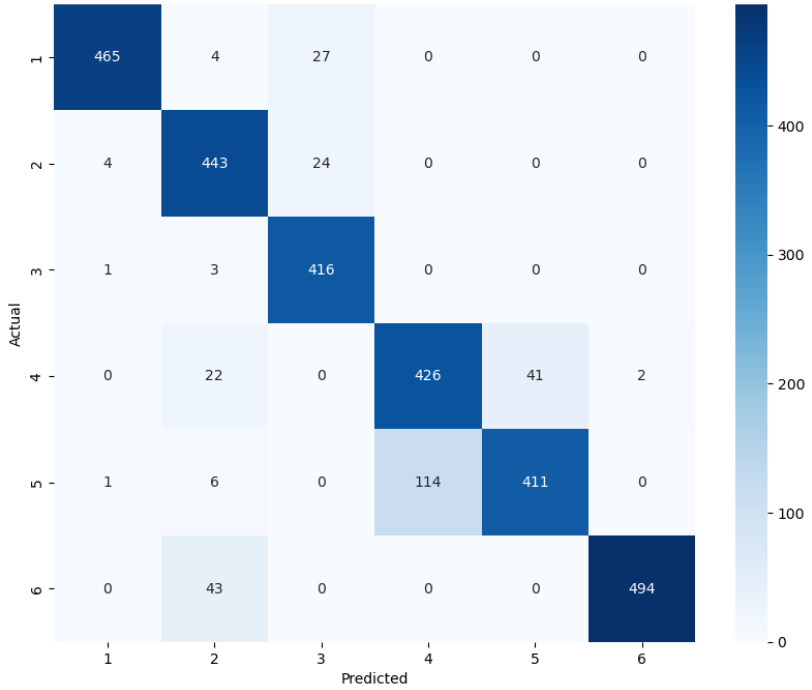
- Building the models:
  - LSTM model
  - CNN model

CNN:

1) Tow 1D CNN layers with 64 filters of kernel size 4
2) followed by a dropout layer (to reduce overfitting)
3) pooling layer before flattening
4) finally, a dense fully connected layer
5) at last we have the softmax layer to represent 6 classes

```
n_timesteps, n_features, n_outputs = trainX.shape[1], trainX.shape[2], trainy.shape[1]
model = Sequential()
model.add(Conv1D(filters=64, kernel_size=4, activation='relu', input_shape=(n_timesteps,n_features)))
model.add(Conv1D(filters=64, kernel_size=4, activation='relu', input_shape=(n_timesteps,n_features)))
model.add(Dropout(0.5))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(100, activation='relu'))
model.add(Dense(n_outputs, activation='softmax'))
plot_model(model, show_shapes=True, show_layer_names=True)
```

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```



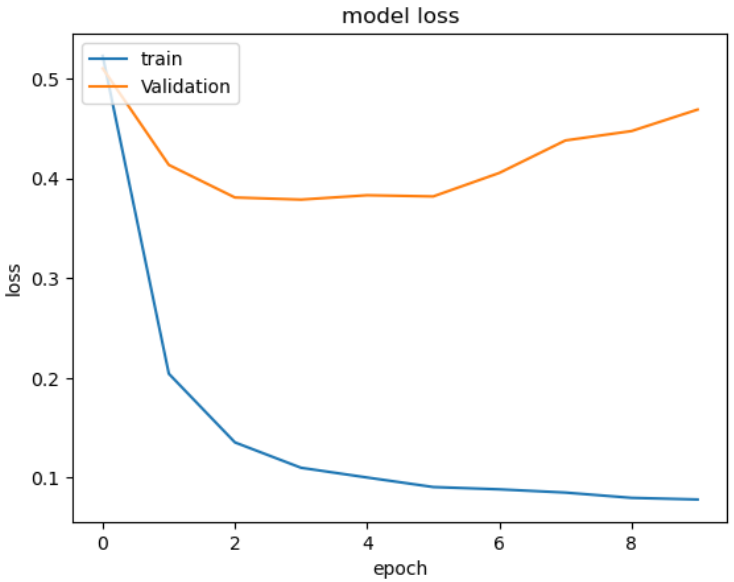
**** Classification Report ****

	precision	recall	f1-score
1	0.99	0.94	0.96
2	0.85	0.94	0.89
3	0.89	0.99	0.94
4	0.79	0.87	0.83
5	0.91	0.77	0.84
6	1.00	0.92	0.96
accuracy			0.90
macro avg	0.90	0.90	0.90
weighted avg	0.91	0.90	0.90

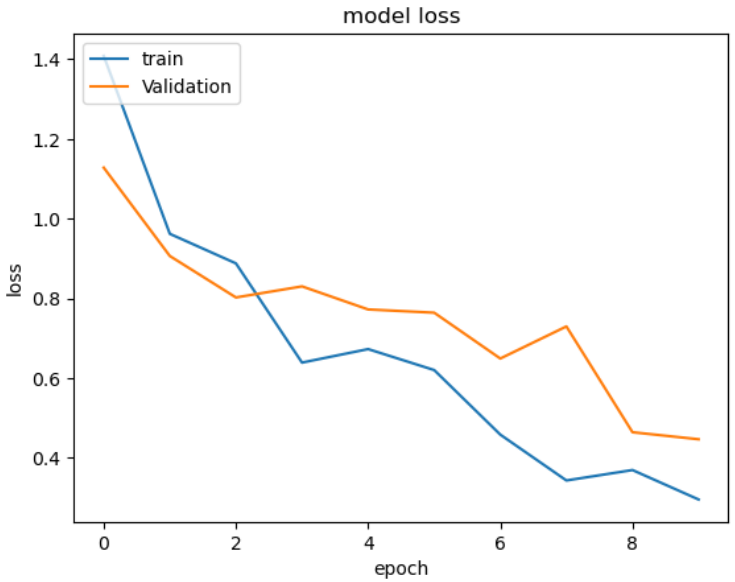
Trained a CNN for 10 epochs

UCI Example:

- Building the models:
  - LSTM model
  - CNN model



CNN model



LSTM model

References

- Gharehbaghi, A. (2023), Deep Learning in Time Series Analysis. CRC Press.
- Chollet, F., (2021), Deep Learning with Python, Manning Publications.
- Brownlee, J. (2018), Deep Learning for Time Series Forecasting: Predict the Future with MLPs, CNNs and LSTMs in Python.
- MIT Introduction to Deep Learning, "Deep Sequence Modeling".
- StatQuest with Josh Starmer, "Recurrent Neural Networks (RNNs), Clearly Explained!!!"
- cs231 Li et al., " Recurrent Neural Networks", 2022.
- CPSC540, "Machine Learning", 2019.
- N. Zolaktaf, "Recurrent Neural Networks", 2016.
- C. C. Aggarwal, Neural Networks and Deep Learning, Chapter 7.1–7.2, IBM T J Watson Research Center Yorktown Heights, NY.